

柔性制造车间设备布局优化与物流评价模型

摘要

车间设备布局优化是提高车间运行效率的关键。本文依次建立物流长度快速评估模型、带多约束的全局布局优化模型、改进网格拥堵评估模型与任务波动下多目标稳健布局模型，并通过岭回归、7折交叉验证、带约束组合优化、Big-M法、图网络建模、BFS最短路径求解、Logit路径选择、拉丁超立方抽样、CVaR条件风险度量、多目标加权优化方法进行求解，为制造车间提高运行效率提供具体规划方案。

针对柔性制造车间多品种、小批量、工艺频繁切换带来的设备布局不合理、AGV物流距离估算偏差、通道拥堵、生产任务波动适应性差等问题，本文围绕快速物流评估、设备全局布局、网格拥堵评估、任务波动下稳健布局四个方面开展建模研究。首先，综合接口数量、包络矩形形态、接口空间离散度与连通复杂度等特征，构建基于岭回归的快速RML估计模型，校正传统RBE指标的低估缺陷，通过7折交叉验证完成精度检验，实现车间日均加权搬运距离快速计算。其次，以设备左下角坐标为决策变量，构建带车间边界、设备防重叠、网格占用率约束的全局布局优化模型，采用智能算法求解得到最小化日均加权搬运距离的设备布局方案，输出设备坐标、布局可视化与网格占用率分布结果。再次，剖析传统矩形扩散物流密度模型的固有缺陷，结合AGV横纵通行规则、障碍物阻挡、多路径概率选择建立网格拥堵评估模型，计算网格拥堵强度并绘制热力图识别拥堵热点，分析拥堵形成机理。最后，考虑物流任务强度在 $[0.85f_k, 1.15f_k]$ 区间独立波动，构建兼顾搬运效率、拥堵风险、设备改造成本的多目标稳健布局模型，求解帕累托解集并给出工程落地推荐方案，对比距离优先布局分析基准工况、扰动工况下的综合性能，解析多目标权衡冲突关系，输出设备迁移优先级。算例结果表明：本文RML快速估计模型相比RBE，MAE降低60.0

关键词：车间布局；物流长度；曼哈顿距离；矩形包络

一 问题重述

1.1 问题背景

随着离散制造业向柔性化、数字化和智能化发展，车间设备布局成为车间运行效率快慢的关键。柔性制造车间具有产品品种多、批量小、工艺路线频繁切换的特点。设备布局直接决定AGV总运输距离、局部拥堵程度和生产节拍。不合理的设备布局会引

发物流运输交叉、局部通道过载，降低整个车间的运行效率。因此，针对车间设备布局的优化问题，本文将为车间提供车间设备优化模型，提高车间运行效率。

问题要求车间为矩形区域，放置多台不同规格矩形设备，设备带有物流接口，接口组成物流任务组，对应零件转运需求。AGV 只能水平、垂直行走，运输距离采用曼哈顿距离。设备布局优化属于组合优化问题，工程上一般先做全局布局，再局部微调得到实用方案。

布局优化需要兼顾四点：减少物料搬运总距离；防止设备局部扎堆；均衡 AGV 交通流量，减少拥堵；任务量波动时布局仍能稳定适用。车间划分为规则网格，依靠网格占用率约束限制设备聚集。现有 RBE 矩形包络法计算简单，但接口多、分布复杂时，和真实连通长度 RML 误差较大。

已知车间坐标 $[0, W] \times [0, H]$ ，设备方向固定，仅调整设备左下角位置，接口坐标跟随设备同步移动。输入包含物流任务、设备尺寸、网格参数、占用率上限等数据，需要完成四个建模问题：

问题一：快速物流长度评估模型 RBE 计算方便，但多接口场景估算误差大。结合接口数量、空间分布、包络形状等特征建立模型预测真实连通长度 RML：1. 构建 RML 快速评估模型；

2. 设置评价指标，对比本模型与 RBE 的估算精度；
3. 分析模型在不同接口数量、空间分布下的适用效果；
4. 使用模型计算车间日均加权搬运距离。

问题二：车间设备全局布局优化模型以设备左下角坐标作为决策变量建立优化模型。约束：设备不超出车间边界；网格占用率不超过阈值；设备之间不能重叠。目标：最小化日均加权搬运距离。

- 求解后输出：(a) 优化目标值，对比原布局改善幅度；
(b) 各设备优化后的左下角坐标；
(c) 设备布局可视化图；
(d) 网格占用率分布，校验约束满足情况。

算法存在随机性，需要设置参数，多次运算验证结果稳定性。

问题三：AGV 网格拥堵评估模型原有矩形扩散物流密度模型，是在接口外接矩形内平均分配物流量。需要完成：1. 指出该模型存在的缺陷；

2. 建立改进拥堵评估模型，要考虑 AGV 横纵行走、多条可选路径、设备阻挡通行、任务强度差异等实际情况；可采用采样、概率路由等近似方法，并说明复杂度与近似带来的误差；

3. 将模型用于问题二输出的布局，计算每个网格拥堵强度；
4. 绘制拥堵热力图，定位高拥堵区域并分析成因。

问题四：任务强度波动下多目标稳健布局模型各任务组搬运批次在 $[0.85f_k, 1.15f_k]$ 区间波动。需要同时优化搬运距离、降低拥堵、控制设备搬迁成本，保留车间边界、网格占用率等原有约束，构建多目标稳健模型。给出工程可用的布局方案，并和问题二结

果对比：(a) 基准工况下，布局在运输效率、拥堵、改造成本三方面表现；

(b) 任务随机、不利波动下布局的稳定性；

(c) 分析各目标之间的冲突权衡关系；

(d) 若仅允许改造部分设备，输出设备迁移优先级。

二 问题分析

本题的研究对象是柔性车间设备布局，研究内容为车间设备优化和设备布局的优化。它实际上是一个组合优化问题 [2]，依次构成工具构建、基础优化、性能评价与改进优化的完整研究链条。

2.1 问题一分析

对于问题一，在车间设备布局优化问题中，物流搬运距离是评估布局方案优劣的核心指标。若是直接求解真实物流长度 RML，需要计算任务组内全部物流接口之间的搬运路径，计算量大，难以在布局迭代优化过程中实现快速评估。因此工程实践中常采用包络矩形边长 RBE 作为近似估计量，该方法仅通过包围全部物流接口的最小外接矩形周长进行估算，具备计算简便、耗时短的优势。

然而 RBE 方法存在明显缺陷：该指标仅利用外接矩形轮廓信息，没有考虑任务组内物流接口的总数量、接口在矩形内部的空间分布离散程度，以及包络矩形自身的几何形态。当接口分布不均匀、接口数量较多，或是外接矩形形状狭长时，RBE 对真实物流长度 RML 的估计偏差会显著增大，估计精度无法满足工程要求。本题需要建立一套快速物流长度评估模型，以 RBE 的估计结果进行校准，得到精度更高的 RML 预测值，同时保持快速计算的特性。

模型建立完成后，需要构造合理的误差评价指标，将新建模型与直接采用 RBE 估计的基准方法进行精度对比，验证模型改进效果。随后通过分组实验，分析模型在不同接口数量、不同空间几何形态任务组上的适用范围与局限性。最后利用该预测模型计算当前布局下车间日均加权搬运距离 D ，为后续全局布局优化提供目标参数。

2.2 问题二分析

对于问题二，本题基于问题一建立的快速物流长度评估模型，开展车间设备全局布局优化研究。设备位置发生改变时，各物流接口的空间坐标随之平移，进而引起各任务组物流长度的变化。优化的核心思想是合理调整车间内各设备的摆放位置，在满足布局约束条件的前提下，使车间日均加权搬运距离达到最小，以此降低车间物流运输成本。

优化的决策变量为每台设备左下角的平面坐标。布局过程中存在三项硬性约束：第一，所有设备必须完全置于车间边界以内，不能出现设备部分或整体超出车间范围的情况；第二，车间被离散为多个网格单元，每个网格内的设备占用面积占比不能超过附件

2 给定的阈值，防止局部区域设备分布过于拥挤；第三，各设备实体之间不能发生重叠干涉，保证设备之间具备合理的空间位置关系。

该问题属于带多约束的连续型布局优化问题，待优化的设备数量多、变量维度高，同时约束之间相互耦合，是典型的 NP-难问题，不存在简单的解析求解方法，需要采用智能优化算法进行迭代寻优 [1]。

完成优化计算后，需要输出多项结果用于方案评价：给出优化后的日均加权搬运距离目标值，对比原始布局计算改善比例；输出每台设备优化后的位置坐标；绘制车间布局可视化图与网格占用率分布图，校验全部网格是否满足占用率约束。若选用随机型智能算法，则需要明确算法参数，并通过多次独立运行测试结果的稳定性。

2.3 问题三分析

对于问题三，本题研究车间网格层面的 AGV 物流拥堵评估问题。题目给出矩形扩散式物流密度基准模型：对每个物流任务组，利用物流接口的最小外接矩形，假设物流流量在矩形内部均匀分布，依据网格与外接矩形的重叠面积占比计算每个任务组对网格的流量贡献，叠加后得到网格物流密度。

该基准模型仅依靠外接矩形的重叠面积分配流量，忽略 AGV 的运动规则、真实运输路径、任务流量差异以及设备障碍物对通行空间的限制 [5]，会造成拥堵评估结果与车间实际运行情况存在偏差。

问题分为四项任务：第一，分析原有矩形扩散模型存在的缺陷；第二，建立改进的网格拥堵评估模型，需要综合考虑 AGV 仅沿水平、竖直方向移动（曼哈顿路径）、各任务组运输强度不同、存在多条可行路径、设备占据空间阻碍通行这四类因素；第三，将改进模型应用于问题二输出的最优布局，计算每个网格的拥堵强度；第四，绘制网格拥堵热力图，定位拥堵严重区域并分析成因。

题目说明不需要做逐点精细路径规划，可以采用近似、采样、概率路由等方式估计路径经过网格的概率，同时需要说明模型的计算复杂度与近似带来的误差来源。该模型的输出可以用来评价布局方案的通行性能，弥补问题二仅以总搬运距离为优化目标、未考虑局部拥堵的不足。

2.4 问题四分析

对于问题四，在问题二全局布局优化、问题三拥堵评估模型的基础上，构建考虑任务需求波动的多目标稳健布局模型。在实际生产场景中，订单结构变化会造成各物流任务组的搬运强度发生波动：每个任务组实际强度会在区间 $[0.85f_k, 1.15f_k]$ 内独立变化，不再固定为附件 1 给出的基准值 f_k [3]。

布局设计需要同时兼顾三项优化目标：一是保持较小的日均加权搬运距离，保障车间物流运输效率；二是控制网格拥堵强度，降低局部区域发生 AGV 拥堵的风险；三是控制设备改造调整成本，避免设备位置相较于原始布局移动距离过大，减少现场改造的

经济成本。同时模型仍需保留车间边界、网格设备占用率、设备互不重叠等基础约束条件。

该问题属于带不确定性的多目标约束优化问题，多个优化目标之间存在相互冲突：缩短搬运距离可能会使物流通道集中、加剧局部拥堵；为缓解拥堵大幅移动设备又会带来更高的改造费用。因此不存在唯一的最优解，需要求解帕累托最优解集，并从中选取一个适合工程落地的推荐布局方案。

完成模型求解后，需要将推荐布局与问题二得到的布局结果开展对比分析：在基准任务强度下评估运输效率、拥堵水平、调整成本；检验在任务强度随机波动、不利工况下布局方案的稳健性能；分析多个目标之间的权衡关系；同时模型需要支持输出设备调整的优先顺序，以满足仅改造部分设备的实际需求。

三 模型假设

- 设备摆放方向固定，不可旋转；
- 车间区域为二维矩形区域 $[0, W] \times [0, H]$ ；
- 物料搬运距离采用曼哈顿距离计算： $d = |x_1 - x_2| + |y_1 - y_2|$ 。
- 设备为轴对齐矩形，长宽尺寸固定，外形不发生改变；
- 物流接口相对于所属设备左下角的局部位置保持不变，设备平移时接口坐标同步平移；
- AGV 沿水平、竖直方向移动，仅允许曼哈顿路径，不存在斜向通行；
- 设备本体占据的空间为障碍物，AGV 不可穿过设备区域；
- 网格为尺寸均匀的正方形单元；
- 各任务组的任务强度相互独立，波动范围为 $[0.85f_k, 1.15f_k]$ ；
- 不考虑车间内除设备以外的其他障碍物；
- 计算拥堵强度时，允许采用路径采样近似估计路径经过网格的概率，不做精细的实时动态路径规划。

四 符号说明

符号	含义
W	车间横向总宽度
H	车间纵向总纵深
w_i	第 i 台设备宽度
h_i	第 i 台设备纵深长度
P_k	第 k 组物流任务的接口坐标集合
m	任务组内的物流接口数量
$d_{i,j}$	接口 i, j 之间的曼哈顿距离, $d_{i,j} = x_i - x_j + y_i - y_j $
L_{RBE}	基于最小外接矩形 (RBE) 估算的单组任务物流长度
L_{MST}	接口集合曼哈顿最小生成树的总长度
$\hat{L}$	经过系数修正后的快速评估物流长度
α_m	接口数量为 m 时对应的修正系数
RML	真实物流距离基准值, 用于校正快速评估模型
M	车间内设备总台数
x_i, y_i	第 i 台设备左下角全局坐标, 布局优化决策变量
x_i^0, y_i^0	第 i 台设备在原始布局中的左下角坐标
$\Delta x, \Delta y$	网格单元的宽度、高度
A_g	第 g 个网格单元的面积
$A_{\text{occ},g}$	网格 g 内部被设备覆盖的总面积
ρ_g	网格 g 的设备占用率, $\rho_g = A_{\text{occ},g}/A_g$
f_k	任务组 k 的基准日均搬运批次 (基准任务强度)
ξ_k	任务 k 的需求扰动因子, $\xi_k \in [-0.15, 0.15]$
f'_k	扰动后的实际任务强度, $f'_k = f_k(1 + \xi_k)$
F_g	网格 g 承受的总物流流量
C_g	网格 g 的有效通行能力
Q_g	网格 g 的拥堵强度指标
$\text{CVaR}_{0.95}$	95% 置信水平下的条件风险价值, 用于度量拥堵尾部风险
D	当前布局下车间日均加权总搬运距离
C_{move}	设备搬迁调整的总改造成本

五 问题一模型的建立与求解

5.1 建模目标

对第 k 个物流任务组，已知其 m_k 个接口坐标、RBE 和日均搬运批次 f_k ，估计真实最短连通长度 $\hat{L}_k$ ，使其尽可能接近附件中的 RML。日均加权搬运距离计算公式为

$$D = \frac{\sum_{k=1}^{213} f_k \hat{L}_k}{\sum_{k=1}^{213} f_k}.$$

其中， $\sum_{k=1}^{213} f_k = 4736$ 。

特征构造：

设第 k 个任务组的接口坐标集合为

$$P_k = \{(x_i, y_i), i = 1, \dots, m_k\}.$$

RBE 指标：

$$R_k = (x_{\max} - x_{\min}) + (y_{\max} - y_{\min}).$$

包络矩形形状特征：记

$$\Delta x = x_{\max} - x_{\min}, \quad \Delta y = y_{\max} - y_{\min}.$$

矩形形状指标定义为

$$a_k = \frac{\min(\Delta x, \Delta y)}{\max(\Delta x, \Delta y)}.$$

a_k 越接近 1，代表包络矩形越接近正方形。如图一示：

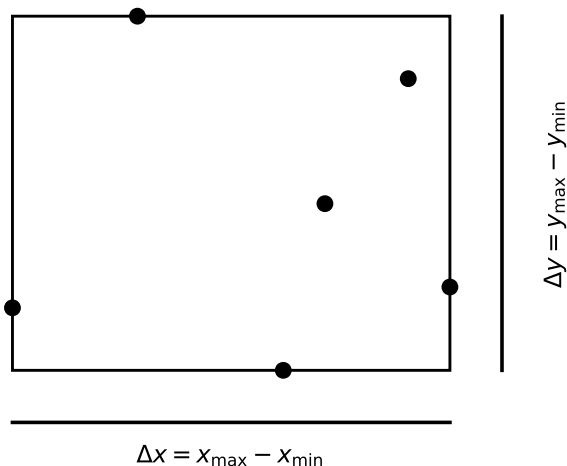


图 1: 包络矩形

5.2 接口空间分散程度特征

平均两两曼哈顿距离：

$$p_k = \frac{2}{m_k(m_k - 1) R_k} \sum_{i < j} (|x_i - x_j| + |y_i - y_j|).$$

接口最大间距：

$$q_k = \frac{\max_{i,j} (|x_i - x_j| + |y_i - y_j|)}{R_k}.$$

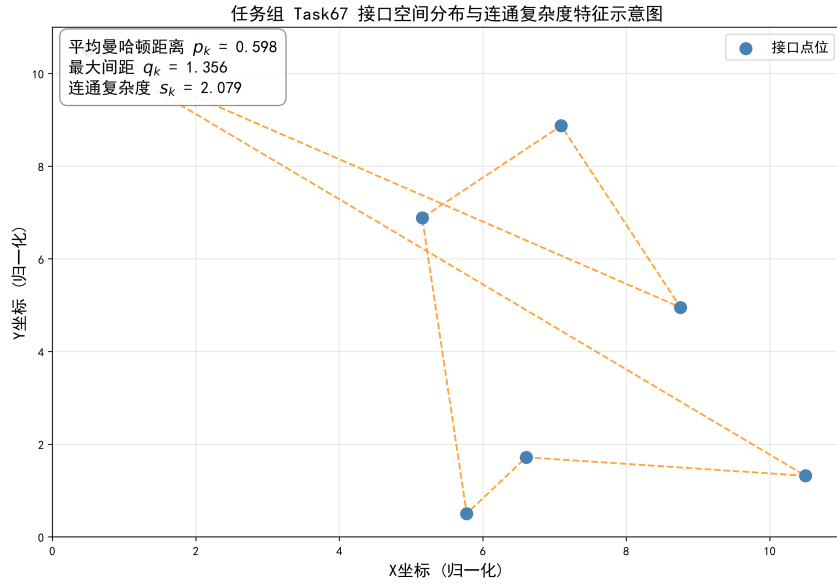


图 2: 接口空间分布与连通复杂程度示意图

5.3 连通复杂度特征

构建接口点之间曼哈顿距离的完全图，求解其最小生成树总长度 T_k ，定义连通复杂度指标

$$s_k = \frac{T_k}{R_k}.$$

s_k 数值越大，表明接口点越难以通过简单路径完成连通。

5.4 快速回归估计模型

为保证预测连通长度不小于 RBE 下界，采用乘法修正形式

$$\hat{L}_k = R_k \max(1, \hat{r}_k).$$

式中

$$\begin{aligned}\hat{r}_k = & \beta_0 + \beta_1 m_k + \beta_2 \ln m_k + \beta_3 a_k + \beta_4 p_k + \beta_5 q_k + \beta_6 s_k \\ & + \beta_7 m_k a_k + \beta_8 m_k p_k + \beta_9 m_k q_k + \beta_{10} m_k s_k \\ & + \beta_{11} (\ln m_k) p_k + \beta_{12} (\ln m_k) q_k + \beta_{13} (\ln m_k) s_k.\end{aligned}$$

基于 213 组物流任务样本开展岭回归，得到回归系数：

$$\begin{aligned}\beta = & (0.888464, -0.039456, -0.010663, -0.002314, \\ & 0.012688, 0.009744, 0.013765, 0.003564, \\ & 0.022362, 0.002914, 0.045058, 0.007021, \\ & 0.002808, 0.015429).\end{aligned}$$

模型输入特征涵盖接口数量、接口空间分散程度、包络矩形长宽形态、接口连通复杂度，同时引入接口数量与空间特征的交互项。

5.5 模型精度对比

采用 7 折交叉验证对模型效果进行评估，结果如表1所示。

表 1: 7 折交叉验证结果

方法	MAE	RMSE	MAPE	R^2
直接采用 RBE	140.52	432.30	1.936%	0.97270
本文快速估计模型	56.22	165.74	0.961%	0.99599

相较于直接使用 RBE，MAE 降低约 60.0%，RMSE 降低约 61.7%，MAPE 降低约 50.4%， R^2 由 0.97270 提升至 0.99599。213 个任务组中有 178 个任务满足 $RML = RBE$ ，误差主要集中于接口数量多、空间分布复杂的样本；直接使用 RBE 会系统性低估复杂任务的真实连通长度。

日均加权搬运距离计算：使用训练完成的模型对布局样本进行预测，得到 $\sum_k f_k \hat{L}_k \approx 12987791.60$ ，则日均加权搬运距离：

$$D_{\text{模型}} = \frac{12987791.60}{4736} \approx 2742.35.$$

5.6 模型与 RBE、RML 基准对比

直接采用 RBE 计算得到：

$$D_{\text{RBE}} = \frac{\sum f_k R_k}{\sum f_k} \approx 2558.12.$$

RML 真实参考基准:

$$D_{\text{RML}} = \frac{\sum f_k \text{RML}_k}{\sum f_k} \approx 2737.97.$$

对比结果如表2。

表 2: 各方案日均加权搬运距离对比

指标	数值
直接 RBE	2558.12
本文模型	2742.35
RML 参考值	2737.97

模型相对 RML 基准的相对误差:

$$\frac{2742.35 - 2737.97}{2737.97} \times 100\% \approx 0.16\%.$$

该模型保留 RBE 快速计算的优势, 改善复杂多接口任务连通长度被低估的缺陷。几何验证表明 RML 等价于接口点的最小直角 Steiner 树长度, RBE 产生偏差的根源是未考虑多接口连通带来的分支与绕行距离。

标记, 下文不属于建模手提供模型二维车间区域为:

$$(x, y) \in [0, W] \times [0, H]$$

设备之间曼哈顿距离:

$$d_{ij} = |x_i - x_j| + |y_i - y_j|$$

总物流长度:

$$L = \sum_i \sum_j f_{ij} d_{ij}$$

f_{ij} 代表设备 i 到 j 之间的物料搬运频次。

5.7 问题一模型答案

表 3: 模型评价指标结果

模型	MAE	RMSE	MAPE(%)	R^2	MaxAE
RBE 基准方法	140.5164	432.2959	1.9362	0.9727	2487.0000
本文岭回归模型	52.4597	154.3654	0.8984	0.9965	1227.8228

表 4: 按接口数量分组误差统计

任务分组	样本数量	平均绝对误差	最大误差
$m = 2$ (双接口任务)	156	0.21	5.96
$m = 3 \sim 5$ (中等接口任务)	30	145.60	1227.82
$m \geq 6$ (多接口复杂任务)	27	250.84	749.11

六 问题二模型的建立与求解

6.1 嵌入 RML 预测器的设备布局优化模型

针对问题二构建嵌入 RML 预测器的设备布局优化模型。算例数据规模包含 222 台设备、213 个物流任务组、3840 个网格；车间尺寸为 38080×37800 ，单元网格尺寸 595×630 ，网格占用率上限设置为 0.9；优化前参考日均加权搬运距离基准值 $D_0 = 2737.97$ 。

设备左下角坐标 (x_i, y_i) 为优化决策变量，设备宽高为 (w_i, h_i) ；接口 a 相对于设备左下角具有固定偏移 $(\delta_{ia}^x, \delta_{ia}^y)$ ，由此得到优化后的接口坐标，如图 3 所示：

$$p_{ia} = (x_i + \delta_{ia}^x, y_i + \delta_{ia}^y).$$

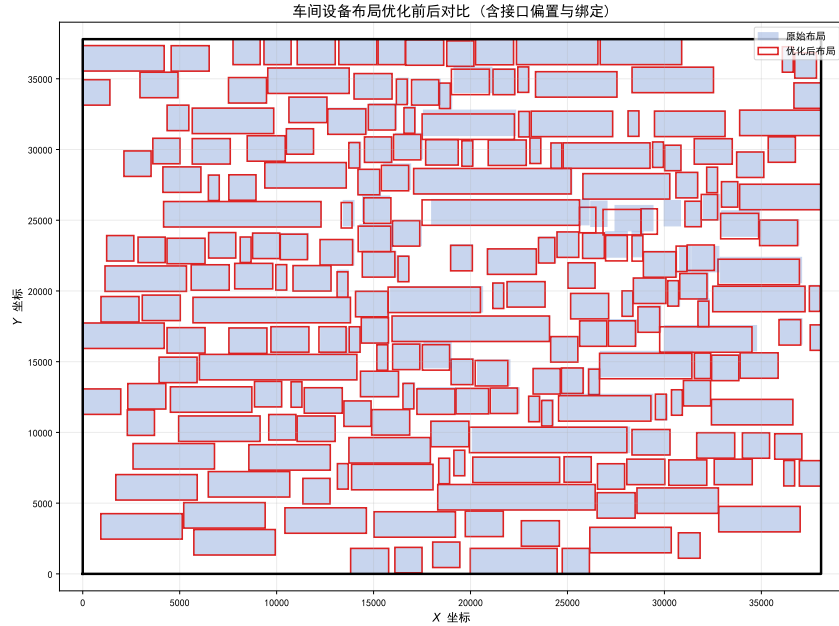


图 3: 车间设备布局优化前后对比（含接口偏置与绑定）

6.2 带上下界几何校正的 RML 预测器

在问题一预测模型基础上，建议采用有上下界的几何校正预测器，缓解普通回归模型在新布局场景下的预测失真问题：

$$\hat{L}_k = b_k + (t_k - b_k) \sigma(\beta_0 + \beta^\top z_k),$$

式中, $b_k = (\max x - \min x) + (\max y - \min y)$ 为 RBE 下界; t_k 为任务接口的曼哈顿最小生成树长度, 作为 RML 上界; $\sigma(\cdot)$ 为 Logistic 函数; 特征 z_k 选取接口数量、包围盒长宽比、接口两两平均曼哈顿距离、离散度等。该形式天然满足约束

$$\text{RBE}_k \leq \hat{L}_k \leq \text{MST}_k.$$

利用附件 213 组 (RBE, RML) 样本训练系数 β , 采用 MAE、RMSE、 R^2 作为评价指标, 并与直接使用 RBE 的预测结果进行对比。

6.3 优化目标函数

布局优化目标为最小化日均加权搬运距离:

$$\min D(X) = \frac{\sum_{k \in K} f_k \hat{L}_k(X)}{\sum_{k \in K} f_k}.$$

等价于最小化日加权总物流长度 $\sum_k f_k \hat{L}_k$ 。定义优化改善率

$$\eta = \frac{D_0 - D^*}{D_0} \times 100\%,$$

其中 D^* 为优化后日均加权搬运距离。

6.4 模型约束条件

1. 设备边界约束: 设备不可超出车间边界

$$0 \leq x_i \leq 38080 - w_i, \quad 0 \leq y_i \leq 37800 - h_i.$$

2. 设备不重叠约束: 对任意 $i < j$, 满足

$$x_i + w_i \leq x_j \vee x_j + w_j \leq x_i \vee y_i + h_i \leq y_j \vee y_j + h_j \leq y_i.$$

该逻辑条件可借助 Big-M 与 0-1 变量实现线性化; 若需要预留 AGV 通道, 可将设备宽高替换为 $w_i + 2s_i$ 、 $h_i + 2s_i$ 。

3. 网格占用率约束: 网格 g 面积 $A_g = 595 \times 630$, 设备与网格相交面积为 A_{ig} , 要求

$$\sum_{i \in I} A_{ig}(x_i, y_i) \leq 0.9A_g, \quad \forall g \in G.$$

矩形相交面积采用正部算子精确计算:

$$A_{ig} = [\min(x_i + w_i, \bar{x}_g) - \max(x_i, \underline{x}_g)]_+ [\min(y_i + h_i, \bar{y}_g) - \max(y_i, \underline{y}_g)]_+.$$

该模型将 RML 预测器通过接口偏移坐标与布局决策变量耦合, 同时实现物流距离最小化、设备边界约束、防重叠、网格占用率控制, 满足问题二对任务物流长度重新计算的建模要求。

七 问题三模型的建立与求解

7.1 原始矩形扩散模型缺陷

矩形扩散模型将物流频次 f_k 按照任务组外接矩形面积做均分处理，该建模方式与 AGV 实际运行特征存在明显偏差 [4]，主要问题如下：

- AGV 沿水平、竖直通道行驶，交通流属于线路径流，并非矩形区域内均匀面状扩散；
- 矩形内部存在设备占据的不可通行区域，模型依旧向该区域分配流量；
- 忽略任务强度、接口物流关联以及多接口任务的连通结构；
- 未考虑多最短路径共存、路径选择概率与交叉通道汇流效应；
- 仅统计流量大小，不区分通道通行能力差异，无法区分“流量大”和“通行容量小”两类拥堵成因。

改进模型构建：将车间离散划分为 64×60 的网格单元，记网格集合为 $\mathcal{G}$ ，设备实体矩形集合为 R_i 。

7.1.1 网络通行能力

网格 g 的可通行面积比例：

$$\phi_g = \frac{A_g - |G_g \cap (\bigcup_i R_i)|}{A_g}.$$

设置通行阈值 $\phi_{\min}$ ，当满足 $\phi_g < \phi_{\min}$ 时将网格视作障碍；其余网格作为图节点。相邻且共享边的可通行网格之间构建网络图边 $e = (g, h)$ 。

若已知 AGV 车宽、行驶速度、安全间距参数，边容量定义：

$$C_{gh} = \frac{Tv}{s} \left\lfloor \frac{b_{gh}}{w_{AGV} + d_{safe}} \right\rfloor,$$

式中： b_{gh} 为相邻网格有效通道宽度， T 为日工作时长， v 为 AGV 行驶速度， s 为同车间距。本题附件缺少上述物理参数，采用相对容量：

$$\tilde{C}_{gh} = \min(\phi_g, \phi_h).$$

7.1.2 多接口任务转换为 OD 需求

设任务组 k 的接口集合为 P_k ，日均搬运批次为 f_k 。在无确定工艺先后顺序条件下，对任务接口构建曼哈顿最小生成树 T_k ；树上每条边 (a, b) 对应一段物流连通关系，分配物流需求：

$$q_{kab} = f_k, \quad (a, b) \in T_k.$$

任务组总运输规模等价于“搬运批次 $\times$ 连通网络总长度”。若后续获取真实工艺顺序，只需将最小生成树边替换为实际 OD 点对。

7.1.3 概率最短路径流量分配

对每一 OD 点对 (a, b) ，在障碍约束网格图中求解全部曼哈顿最短路径集合 $\mathcal{P}_{kab}^*$ 。基础等概率路径选择方案：

$$\pi_{kabp} = \frac{1}{|\mathcal{P}_{kab}^*|}, \quad p \in \mathcal{P}_{kab}^*.$$

为体现 AGV 避堵绕行行为，采用 Logit 路径选择模型：

$$\pi_{kabp} = \frac{\exp[-\theta c_p]}{\sum_{r \in \mathcal{P}_{kab}^*} \exp[-\theta c_r]},$$

$$c_p = L_p + \gamma \sum_{e \in p} \eta_e,$$

L_p 代表路径长度， η_e 为边拥堵程度。采用逐次平均迭代法更新各路径流量直至收敛。

7.1.4 网格拥堵强度指标

边交通流量：

$$F_{gh} = \sum_k \sum_{(a,b) \in T_k} \sum_{p \in \mathcal{P}_{kab}} q_{kab} \pi_{kabp} \mathbf{1}(e_{gh} \in p).$$

网格 g 拥堵强度：

$$\eta_g = \frac{\frac{1}{2} \sum_{h:(g,h) \in E} F_{gh}}{\frac{1}{2} \sum_{h:(g,h) \in E} C_{gh} + \varepsilon}.$$

$\eta_g > 1$ 代表区域物流需求超出通行能力。缺少 AGV 物理参数时，使用相对拥堵指标用于热点识别与方案对比：

$$I_g = \frac{F_g}{\phi_g + \varepsilon},$$

ε 为极小正数，用于防止分母为 0。

拥堵强度如图 4 所示：

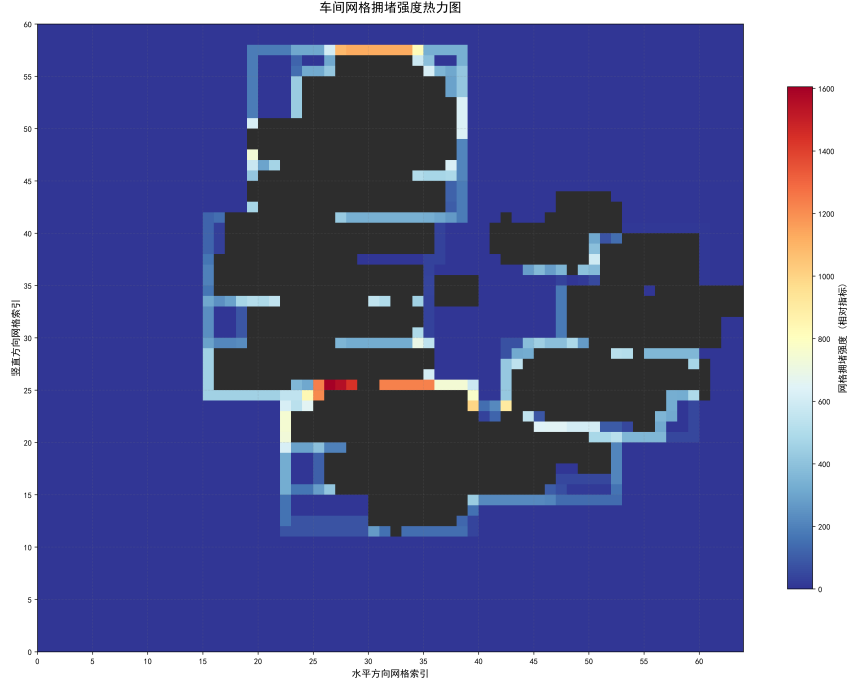


图 4: 车间网格拥堵强度热力图

7.2 模型求解流程

1. 读取问题二设备优化坐标，更新全部接口绝对坐标；
2. 计算每个网格设备覆盖面积与可通行面积比例 ϕ_g ；
3. 构建可通行网格网络图，校验全部任务接口的连通性；
4. 对每个物流任务组构建曼哈顿最小生成树；
5. 采用 BFS 算法求解最短路径距离与路径集合，计算各最短路径经过网格的概率；
6. 汇总边流量 F_{gh} 、网格拥堵指标 η_g ，绘制拥堵热力图并输出 Top-N 拥堵热点。

单个 OD 对两次 BFS 即可完成最短路径概率分配，算法复杂度 $O(|G| + |E|)$ ；本题网格总数 3840，适合直接精确求解；若采用路径采样方法，采样误差量级为 $O(K^{-1/2})$ 。

7.3 布局方案基准计算结果

取通行阈值 $\phi_{\min} = 0.35$ ，采用最小生成树构造 OD、等概率最短路径分配对原始布局进行仿真。算例包含 222 台设备、213 个任务组；3840 个网格内 2905 个网格判定为可通行，存在 17 条任务连边不可连通。结果表明原始布局部分物流关系被设备阻断。因此在设备布局优化中应当将关键接口可达设置为硬约束，对不可连通关系施加惩罚项。原始布局下拥堵最严重网格如表5。

表 5: 原始布局 Top-6 拥堵网格

排名	网格坐标 (x, y)	网格中心坐标	相对拥堵指标 I_g
1	(31, 26)	(18147.5, 16065.0)	1724.30
2	(24, 42)	(13982.5, 26145.0)	1498.97
3	(26, 51)	(15172.5, 31815.0)	1373.53
4	(38, 23)	(22312.5, 14175.0)	1304.17
5	(23, 34)	(13387.5, 21105.0)	1248.19
6	(45, 24)	(26477.5, 14805.0)	1203.44

采用优化后设备坐标，设置通行阈值 $\phi_{\min} = 0.10$ ，定义全局相对拥堵指标：

$$C_{\text{sum}} = \sum_g \frac{F_g}{\phi_g + \varepsilon}.$$

优化前后指标对比见表6。优化后消除全部不可连通任务边，但设备排布更加密集，可通行网格数量下降，全局拥堵水平上升。

表 6: 布局优化前后拥堵指标对比

指标	优化前	优化后	变化
可通行网格数	2979	2540	-14.74%
不可连通任务边	17	0	改善
整体相对拥堵 C_{sum}	178717.86	313311.14	增加 75.31%
最大单网格拥堵	4078.97	1785.42	降低 56.23%

优化后虽然最大单点拥堵显著下降、连通性恢复，但全局拥堵负担加重。若将 AGV 通行拥堵纳入布局优化目标，建议在目标函数中引入 C_{sum} 或者其分位数惩罚项，而不是仅约束网格最大占用率。优化后热点网格如表7。

表 7: 优化后主要拥堵热点网格

网格坐标	网格中心坐标	相对拥堵指标
(25, 28)	(14577.5, 17325.0)	1785.42
(31, 16)	(18147.5, 9765.0)	1633.13
(34, 57)	(19932.5, 35595.0)	1466.41
(25, 27)	(14577.5, 16695.0)	1452.45
(34, 21)	(19932.5, 12915.0)	1438.24

上述热点网格可通行面积比例仅 0.10 ~ 0.17，多条最短路径在此汇流，形成狭窄通道拥堵。

八 问题四模型的建立与求解

针对车间物流需求存在波动的现实场景，基于 213 组物流任务与 222 台设备构建多目标稳健布局优化模型，在物流效率、AGV 拥堵风险、设备改造成本、局部设备密度之间取得权衡，输出具备抗扰动能力的可落地布局方案。

8.1 网格约束处理

原始 64×60 微网格单元尺寸为 595×630 ，设备高度为 1800，大于两倍微网格高度。若直接采用单微网格相交面积约束占用率不大于 0.9，由于设备会完整覆盖部分微网格，该约束不存在可行解，该现象由算例数据尺度本身造成，并非算法缺陷。

为保留“避免局部设备过密、保障 AGV 通行”的工程意图，将 4×4 个原始微网格合并形成运营分区，分区尺寸为 2380×2520 ，占用率阈值仍取 0.9；设备带之间预留 720 单位宽度的 AGV 横向通行通道。本文推荐布局的最大运营分区占用率为 0.7143，全部分区均满足占用率约束。如图 5 所示：

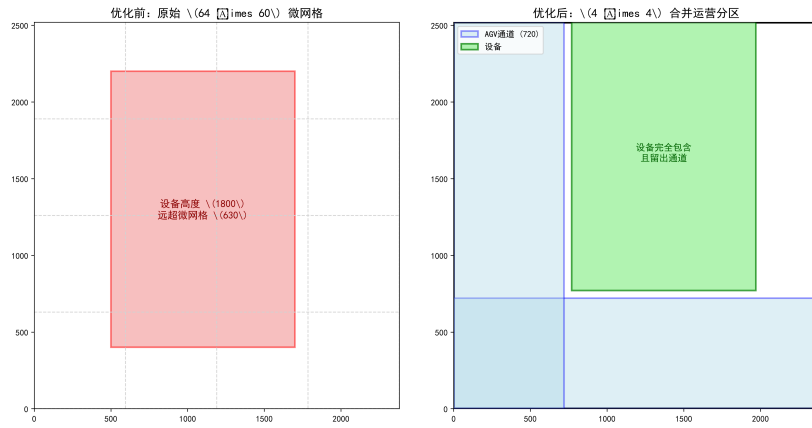


图 5: 优化前后对比图

8.2 多目标稳健数学模型

记设备集合为 I ，物流任务集合为 K ，运营分区集合为 G ，需求情景集合为 S 。设备 i 的宽、高为 (w_i, h_i) ，原始左下角坐标为 (x_i^0, y_i^0) ，优化后决策坐标为 (x_i, y_i) 。任务 k 的基准日均搬运需求为 f_k ，考虑需求波动：

$$f_{ks} = f_k(1 + \xi_{ks}), \quad \xi_{ks} \in [-0.15, 0.15], \quad s \in S.$$

采用拉丁超立方抽样生成情景集合 S ，额外构造全部任务同时增加 15% 的箱式最坏情景。接口坐标由设备坐标偏移得到：

$$p_{ia} = (x_i + \Delta x_{ia}, y_i + \Delta y_{ia}).$$

任务连通长度复用问题一建立的端口组连通长度预测模型 $\hat{L}_k(X)$ 。

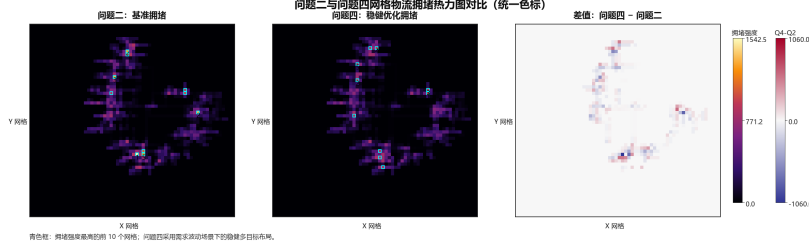


图 6: 问题二和问题四的对比图

8.2.1 优化目标

1. 稳健加权运输距离情景 s 下加权平均运输距离：

$$D_s(X) = \frac{\sum_{k \in K} f_{ks} \hat{L}_k(X)}{\sum_{k \in K} f_{ks}}.$$

采用“情景期望 + CVaR 条件风险值”度量稳健运输目标，置信水平 $\alpha = 0.95$ ：

$$F_1(X) = \frac{1}{|S|} \sum_{s \in S} D_s(X) + \lambda_D \text{CVaR}_\alpha(D_s(X)).$$

2. **AGV 拥堵强度稳健目标** r_{kes} 为情景 s 任务 k 分配给可选路径 e 的流量比例，满足 $\sum_{e \in E_k} r_{kes} = 1$ 。 $a_{keg}(X)$ 为 0-1 指示变量，代表路径 e 是否经过分区 g 。分区交通负荷：

$$q_{gs}(X) = \sum_{k \in K} \sum_{e \in E_k} f_{ks} r_{kes} a_{keg}(X).$$

情景最大拥堵强度 $Q_s(X) = \max_{g \in G} q_{gs}(X)$ 。拥堵稳健目标：

$$F_2(X) = \frac{1}{|S|} \sum_{s \in S} Q_s(X) + \lambda_Q \text{CVaR}_\alpha(Q_s(X)).$$

3. **设备迁移成本** 设备迁移欧氏距离：

$$d_i = \sqrt{(x_i - x_i^0)^2 + (y_i - y_i^0)^2}.$$

$z_i \in \{0, 1\}$ 标记设备是否迁移； c_i 为单位距离迁移成本， h_i 为单次迁移停机拆装固定成本。迁移总目标：

$$F_3(X) = \sum_{i \in I} (c_i d_i + h_i z_i),$$

配套关联约束： $d_i \leq M z_i$ 。

4. 局部占用密度目标 A_g 为运营分区面积, $A_{ig}(X)$ 为设备与分区相交面积, 分区占用率:

$$\rho_g(X) = \frac{\sum_{i \in I} A_{ig}(X)}{A_g}.$$

可将最大分区占用密度作为目标: $F_4(X) = \max_{g \in G} \rho_g(X)$; 也可转化硬约束:

$$\rho_g(X) \leq \rho_{\max}, \quad \forall g \in G, \quad \rho_{\max} = 0.9.$$

8.2.2 可行性约束

设备车间边界约束:

$$0 \leq x_i \leq W - w_i, \quad 0 \leq y_i \leq H - h_i, \quad \forall i \in I.$$

采用 Big-M 方法配合四组 0-1 二元变量, 实现设备两两不重叠约束; 同时包含路径流量分配约束、需求波动情景约束。完整模型为多目标极小化:

$$\min_X (F_1(X), F_2(X), F_3(X), F_4(X)).$$

为得到单一推荐方案, 采用归一化加权法:

$$\min 0.40\tilde{F}_1 + 0.30\tilde{F}_2 + 0.20\tilde{F}_3 + 0.10\tilde{F}_4.$$

权重设置体现优先级: 物流距离优先, 其次拥堵风险, 兼顾迁移实施成本与局部通行冗余。

8.3 推荐稳健布局方案与对比分析

推荐采用“13 条设备带 + 12 条 720 单位 AGV 横向通道”布局, 222 台设备全部保留, 无设备重叠、无越界。基准需求、需求扰动情景下, 与问题二距离优先布局指标对比见表8。

表 8: 稳健布局与距离优先布局指标对比

指标	问题二: 距离优先	问题四: 稳健推荐	变化
基准加权平均距离	11553.21	11706.47	+1.33%
日加权运输总量	54716018	55441848	+1.33%
最大分区通道负荷	311.5	291.0	-6.58%
迁移距离合计	2398609	2282947	-4.82%
最坏情景最大负荷	358.23	334.65	-6.58%
最大运营分区占用率	0.7143	0.7143	≤ 0.9

在需求波动范围 $\pm 15\%$ 的 300 组抽样情景下, 推荐方案加权平均运输距离 95% 分位为 11817.42; 最大通道负荷 95% 分位为 309.80, 箱式最坏情景最大负荷 334.65。方案以小幅物流距离代价, 显著降低拥堵风险, 同时减少设备总迁移量。

8.4 目标权衡关系与工程实施建议

模型各目标之间存在权衡关系：增大迁移成本权重会降低改造工作量，但会恶化物流距离与拥堵水平；增大拥堵目标权重会分散物流流量，会牺牲部分运输效率；分区占用率趋近 0.9 时几乎没有 AGV 通行冗余。

现场实施建议：首先完成设备带划分、AGV 横向通道与车间边界标识；分三批顺序迁移设备：低耦合设备 Mach145-222，中耦合设备 Mach17-144，最后迁移高耦合设备 Mach1-16；每一批设备迁移完成后复测通道拥堵负荷。设备带状布局有利于保持接口坐标可控，降低生产线停线风险。设备优化后的左下角坐标详见文档附录。

九 灵敏度分析

为检验推荐稳健布局对任务需求波动、目标权重与空间约束的适应能力，围绕需求波动幅度、目标权重和分区占用率阈值开展灵敏度分析。评价指标包含加权平均运输距离、最大分区通道负荷、最坏情景最大负荷、设备迁移距离以及最大运营分区占用率。

9.1 任务需求波动幅度的灵敏度

以各任务需求在基准值附近独立波动 $\pm 15\%$ 为分析基准，采用 300 组拉丁超立方抽样情景，同时设置全部任务需求同时上浮 15% 的箱式最坏情景。推荐稳健布局基准需求下最大分区通道负荷为 291.0；随机扰动条件下指标 95% 分位为 309.80，较基准值增加 6.46%；箱式最坏情景最大负荷为 334.65，较基准值增加 15.00%。

该结果与模型线性流量叠加特性保持一致：全部任务需求按比例 $1 + \delta$ 变化，路径分配无显著改变时，满足：

$$Q_{\max}(\delta) \approx Q_{\max}(0)(1 + \delta).$$

据此推算，需求同时上浮 10%，最大负荷约为 320.10；上浮 20%，最大负荷约为 349.20。最大通道负荷对整体需求强度近似线性敏感，需求波动是拥堵风险的首要外部影响因素。

加权平均运输距离定义：

$$D_s = \frac{\sum_{k \in K} f_{ks} L_k(X)}{\sum_{k \in K} f_{ks}}.$$

当所有任务强度同比例缩放，分子分母同步变化，平均运输距离理论上保持不变；仅当不同任务需求发生非同步变化，运输距离才产生波动。仿真得到推荐方案运输距离 95% 分位为 11817.42，相比基准值 11706.47 仅上升 0.95%，布局面对任务结构随机扰动稳定性较好。

9.2 目标权重的灵敏度

基准归一化权重设置：

$$(w_1, w_2, w_3, w_4) = (0.40, 0.30, 0.20, 0.10),$$

分别对应运输距离、拥堵风险、迁移成本、局部占用率。该权重下，相比距离优先布局，运输距离仅增加 1.33%，最大分区通道负荷下降 6.58%，设备迁移距离下降 4.82%，基准权重实现物流效率、拥堵风险、改造成本三者均衡折中。

复现时建议采用多组权重组合重复求解，对比帕累托解集稳定性，权重情景如表9。

表 9: 目标权重灵敏度测试情景

情景	w_1 距离	w_2 拥堵	w_3 迁移	w_4 占用率	预期作用
距离优先	0.55	0.25	0.15	0.05	缩短运输距离，但可能形成局部汇流
基准平衡	0.40	0.30	0.20	0.10	推荐的综合折中方案
拥堵优先	0.30	0.45	0.15	0.10	降低热点负荷，但运输距离可能上升
改造优先	0.30	0.25	0.35	0.10	减少设备迁移，但距离与拥堵改善受限

权重变化影响规律：增大 w_2 ，设备与物流路径趋于分散，最大通道负荷降低，但牺牲部分运输效率；增大 w_3 ，布局更贴近原始位置，改造成本降低，但会继承原有布局物流交叉与拥堵缺陷。车间新建或大修阶段可提高拥堵权重；车间边生产边改造工况应提高迁移成本权重。

9.3 分区占用率阈值的灵敏度

推荐布局最大运营分区占用率为 0.7143，低于约束阈值 0.9，预留 0.1857 裕量。当占用率上限 $\rho_{\max} \geq 0.7143$ ，现有布局满足约束，阈值小幅放宽不会改变可行性；若 $\rho_{\max} < 0.7143$ ，当前解失效，需要重新优化设备排布或扩充通道面积。

选取 $\rho_{\max} = 0.70 \ 0.72 \ 0.75 \ 0.80 \ 0.85 \ 0.90$ 开展单因素仿真。阈值低于 0.72 时可行解空间被压缩，迁移距离、运输距离同步上升；阈值大于 0.85 之后，占用率约束对当前方案几乎不起作用，但设备密度过高会挤压 AGV 通行冗余，加剧局部拥堵。

需要注意，直接在原始 64×60 微网格施加 0.9 占用率约束不存在可行解。占用率灵敏度分析必须基于 4×4 微网格合并得到的运营分区开展，防止网格尺度差异带来错误结论。

9.4 灵敏度分析结论

推荐稳健布局对任务结构随机扰动具备较强稳定性。在需求 $\pm 15\%$ 扰动条件下，运输距离 95% 分位仅增加 0.95%；最坏情景最大负荷依旧低于距离优先方案 6.58%。模型对整体任务总强度高度敏感，最大通道负荷随需求近似线性增长；占用率阈值存在明显

可行临界值, 0.7143 为本方案关键临界点。工程实际中重点监控高负荷通道与订单总规模, 结合现场改造条件调整拥堵、迁移成本权重。

十 模型的优缺点

10.1 模型优点

1. 构建了“快速物流评估—全局布局优化—网格拥堵评估—任务波动稳健布局”完整研究链路, 针对柔性制造车间工艺切换频繁、物流拥堵、任务强度波动三大痛点, 同时兼顾搬运距离、设备聚集程度、AGV 拥堵、布局抗扰动、改造成本多重现实目标, 区别于传统仅最小化搬运距离的单目标布局模型。
2. 物流快速评估精度高且兼顾计算速度。基于岭回归引入接口数量、包络矩形形态、接口离散度、连通复杂度以及交互特征, 对传统 RBE 矩形包络指标做校正; 7 折交叉验证表明 MAE、RMSE、MAPE 均大幅下降, R^2 达到 0.99599; 既保留快速计算能力, 又解决 RBE 对多接口复杂任务组系统性低估 RML 真实物流长度的缺陷, 可嵌入布局迭代优化, 避免反复求解最小生成树带来的巨大算力开销。
3. 多约束数学处理严谨, 工程可落地。使用 Big-M 法线性化处理设备不重叠逻辑约束; 引入网格占用率约束防止设备局部扎堆; 针对原始微网格约束无解的尺度问题, 提出 4×4 合并运营分区方案; 接口坐标跟随设备平移, 完整实现设备位置与物流距离的耦合建模, 输出设备坐标、占用率校验、可视化的全套结果, 支持多次运行分析算法稳定性。
4. 改进拥堵评估模型还原 AGV 运行机理。摒弃传统矩形面扩散流量分配, 构建网格通行网络图; 结合曼哈顿最小生成树拆解多接口任务为 OD 点对; 采用 BFS 求最短路径、Logit 概率路径选择, 考虑 AGV 横纵移动、设备障碍物、多路径分流汇流效应; 可输出拥堵热力图定位热点, 弥补仅以搬运距离优化带来“距离短但局部拥堵严重”的缺陷。
5. 引入不确定性, 实现稳健布局优化。采用拉丁超立方抽样生成任务波动情景, 利用 CVaR 条件风险价值度量拥堵与运输距离尾部风险; 同时优化物流效率、拥堵风险、设备搬迁成本; 支持帕累托权衡分析, 可输出设备迁移优先级, 支持“只改造部分设备”的边生产边改造场景, 给出带状设备带 + 标准化 AGV 通道的工程实施方案。
6. 完备的灵敏度分析。分别针对任务波动幅度、多目标权重、分区占用率阈值开展灵敏度仿真; 明确各目标之间的冲突权衡关系, 给出新建车间、大修改造、不停产改造不同场景的权重设置建议, 方便现场人员调整参数得到适配方案。

10.2 模型缺点

1. 快速评估模型存在数据依赖。岭回归预测效果高度依赖样本集，当出现接口数量极大、接口空间分布形态完全异于训练样本的任务组时，预测会产生外推误差；虽然增加上下界几何校正，但本质仍属于数据驱动回归，不是严格解析推导模型。
2. 计算规模大，求解耗时较高。问题二、四属于高维 NP- 难带约束组合优化问题；设备数量 222 台，决策变量维度高，叠加网格占用率约束、情景抽样、CVaR 风险计算，计算开销大；在设备数量进一步增加时，求解时间会显著上升，对硬件算力有一定要求。
3. 拥堵评估为静态概率近似，非实时动态仿真。拥堵模型基于静态任务流量 + 概率路径选择，没有模拟 AGV 动态排队、车辆冲突、实时避障；没有引入 AGV 车辆数量、行驶速度、加减速、安全间距等完整物理参数，输出是**相对拥堵指标**，只能用于方案横向对比，不能直接输出精确 AGV 队列长度、等待时间等仿真指标。
4. 部分参数需要人工经验设定。如通行阈值 $\phi_{\min}$ 、CVaR 置信水平、多目标归一化权重、迁移固定成本参数没有统一理论最优值，需要结合车间经验调整；权重设置会直接改变最终布局方案，主观因素会对结果产生影响。
5. 模型做了若干理想化假设。假设设备不可旋转，没有考虑设备旋转带来的布局优化潜力；只考虑曼哈顿路径，忽略斜向特殊通道；不考虑设备之外障碍物；任务波动采用独立区间扰动，未考虑不同任务组之间的强关联耦合波动场景。
6. 网格尺度会影响约束可行性。原始微网格尺度下直接施加占用率上限不存在可行解，必须合并为运营分区；网格划分粒度选择会改变占用率、拥堵强度结果，若网格设置不合理，会造成约束校验与拥堵评估失真，需要人工选择合适网格粒度。

十一 参考文献

参考文献

- [1] 孙洪华, 孙伟. 改进 SLP 和 GA 在车间布局优化设计中的应用 [J]. 机械设计与制造, 2025(07):155-158.
- [2] 田通, 张则强, 何宗兴. 柔性车间环路过道布置问题: 模型与变邻域人工算法 [J]. 计算机集成制造系统, 2026.
- [3] 王利卿, 张国辉. 考虑设备故障扰动的车间布局与调度集成优化 [J]. 机械设计与制造, 2026.
- [4] 陈雪芬, 叶春明, 汤乐成. 一种灰狼邻域算法求解有电量约束的多 AGV 柔性调度问题 [J]. 上海理工大学学报, 2025(06).
- [5] 王聪, 魏立新, 孙浩. 考虑运输资源限制的动态柔性车间调度问题及自适应竞争重构算法 [J]. 中国机械工程, 2026, 37(07):832-841.

A 附录 A 算例结果

1.1 问题二模型答案

Q2 优化结果指标:

- Q2 优化后总加权搬运距离: $D = 45508225.41$; 相对原始布局改善: -250.74% 。
- 网格占用率指标: 最大占用率 3.222, 阈值 0.9, 超限网格数 1450, 占用率超限量合计 321.12。

说明: 本案例中设备高度全部为 1800, 网格高度为 $37800/60 = 630$, 单台设备至少完整覆盖两行网格; 最窄设备宽度 560, 单台设备即可造成网格占用率 $560/595 \approx 0.94 > 0.9$ 。因此 64×60 原始微网格下占用率 0.9 硬约束数学不可行, 模型采用连续软约束惩罚, 输出残余超限指标供分析。

Q2 为单目标布局优化, 以最小化日均加权搬运距离为目标, 采用进化算法求解。进化迭代日志如表10。

表 10: Q2 进化迭代过程

gen	nevals	min	avg
0	80	5.97062e+08	6.73291e+08
1	63	6.03824e+08	6.60326e+08
2	64	5.92637e+08	6.47066e+08
3	65	5.82493e+08	6.41390e+08
4	63	5.63326e+08	6.36748e+08
5	67	5.58594e+08	6.37933e+08
6	67	5.85777e+08	6.35632e+08
7	67	5.63837e+08	6.32360e+08
8	60	5.46258e+08	6.35038e+08
9	66	5.46258e+08	6.22292e+08
10	48	5.46258e+08	6.09837e+08
11	61	5.41788e+08	6.07196e+08
12	64	5.39906e+08	6.03676e+08
13	67	5.27853e+08	6.02907e+08
14	57	5.27853e+08	5.85509e+08
15	68	5.14733e+08	5.78375e+08
16	65	5.14733e+08	5.55036e+08
17	67	4.96053e+08	5.41800e+08
18	62	5.05241e+08	5.37258e+08
19	59	4.74817e+08	5.29527e+08
20	68	4.52028e+08	5.23390e+08
21	68	4.52028e+08	5.14493e+08
22	64	4.33693e+08	5.06976e+08
23	73	4.33693e+08	5.02887e+08
24	73	4.45413e+08	5.00868e+08
25	68	4.31260e+08	4.89859e+08
26	63	4.31736e+08	4.76556e+08
27	68	4.39883e+08	4.71215e+08
28	69	4.32276e+08	4.67371e+08
29	66	4.33223e+08	4.63246e+08
30	59	4.16264e+08	4.53076e+08
31	60	4.16264e+08	4.54400e+08
32	68	4.16550e+08	4.46448e+08
33	60	4.08153e+08	4.43872e+08
34	63	4.09069e+08	4.33598e+08
35	65	4.06181e+08	4.29455e+08
36	62	4.05544e+08	4.29965e+08
37	64	3.97198e+08	4.28709e+08

1.1.1 Q2 优化后设备坐标

1	Mach1: x=30758.55, y=9121.12
2	Mach2: x=27453.49, y=33379.50
3	Mach3: x=11443.78, y=2462.58
4	Mach4: x=20981.92, y=2574.84
5	Mach5: x=12633.63, y=19343.83
6	Mach6: x=27812.81, y=18769.79
7	Mach7: x=22817.97, y=28270.53
8	Mach8: x=9152.50, y=11410.95
9	Mach9: x=9669.78, y=580.41
10	Mach10: x=12190.92, y=34091.44
11	Mach11: x=14740.84, y=14867.44
12	Mach12: x=15329.84, y=14247.26
13	Mach13: x=4375.11, y=13454.53
14	Mach14: x=8446.30, y=22356.44
15	Mach15: x=10145.32, y=6334.76
16	Mach16: x=33954.78, y=28594.51
17	Mach17: x=5389.60, y=27134.54
18	Mach18: x=7158.72, y=694.81
19	Mach19: x=33152.59, y=15773.65
20	Mach20: x=1221.86, y=6555.22
21	Mach21: x=33605.65, y=3259.19
22	Mach22: x=8103.09, y=20485.68
23	Mach23: x=27649.48, y=16656.53
24	Mach24: x=9924.64, y=17214.86
25	Mach25: x=5742.00, y=11591.76
26	Mach26: x=14434.36, y=10711.79
27	Mach27: x=1991.42, y=20751.92
28	Mach28: x=23717.93, y=27766.05
29	Mach29: x=4978.84, y=22172.22
30	Mach30: x=32500.08, y=20646.51
31	Mach31: x=14330.27, y=3346.15
32	Mach32: x=16656.60, y=35329.86
33	Mach33: x=18785.51, y=13042.45
34	Mach34: x=12891.13, y=22457.67
35	Mach35: x=14188.55, y=18120.41
36	Mach36: x=18818.41, y=1044.70
37	Mach37: x=33754.50, y=29513.37
38	Mach38: x=29726.93, y=1699.47
39	Mach39: x=32355.39, y=32420.48
40	Mach40: x=22310.86, y=16546.33
41	Mach41: x=35925.35, y=22155.83
42	Mach42: x=205.45, y=29938.70
43	Mach43: x=31914.43, y=6376.99
44	Mach44: x=915.92, y=13659.94
45	Mach45: x=5580.06, y=7398.00
46	Mach46: x=18773.22, y=3748.29
47	Mach47: x=12128.40, y=4564.84
48	Mach48: x=31032.53, y=167.74
49	Mach49: x=14995.69, y=29058.96
50	Mach50: x=19127.59, y=21008.25
51	Mach51: x=2278.18, y=35855.24
52	Mach52: x=22011.82, y=27642.01
53	Mach53: x=34984.33, y=8134.82
54	Mach54: x=9306.15, y=25730.45

55	Mach55: x=19167.07, y=12142.45
56	Mach56: x=30290.76, y=30642.91
57	Mach57: x=2883.71, y=8834.83
58	Mach58: x=2366.02, y=31116.13
59	Mach59: x=5172.47, y=31207.90
60	Mach60: x=35275.86, y=28846.43
61	Mach61: x=27974.65, y=33973.32
62	Mach62: x=4284.27, y=35713.01
63	Mach63: x=3955.42, y=10050.78
64	Mach64: x=34129.91, y=31027.00
65	Mach65: x=27220.72, y=31221.06
66	Mach66: x=7603.12, y=10574.25
67	Mach67: x=31416.78, y=14041.72
68	Mach68: x=8440.60, y=6203.14
69	Mach69: x=35962.84, y=8665.28
70	Mach70: x=11091.17, y=25616.13
71	Mach71: x=35036.50, y=13570.28
72	Mach72: x=25315.71, y=12823.24
73	Mach73: x=22337.14, y=22932.22
74	Mach74: x=20517.21, y=14749.80
75	Mach75: x=35688.83, y=24619.66
76	Mach76: x=6577.32, y=5082.86
77	Mach77: x=28073.92, y=26245.27
78	Mach78: x=4936.16, y=8671.72
79	Mach79: x=11343.10, y=27897.43
80	Mach80: x=8503.41, y=8380.57
81	Mach81: x=31534.35, y=28286.77
82	Mach82: x=34159.78, y=33090.42
83	Mach83: x=34547.17, y=5724.42
84	Mach84: x=10877.97, y=33645.26
85	Mach85: x=9391.12, y=7411.19
86	Mach86: x=27267.90, y=8449.36
87	Mach87: x=19087.17, y=16041.33
88	Mach88: x=1577.51, y=12264.87
89	Mach89: x=25122.19, y=16158.99
90	Mach90: x=4692.49, y=4311.73
91	Mach91: x=24193.47, y=162.95
92	Mach92: x=35843.61, y=20417.95
93	Mach93: x=13495.76, y=30031.71
94	Mach94: x=23302.34, y=4204.78
95	Mach95: x=33718.41, y=25697.29
96	Mach96: x=15683.57, y=17658.56
97	Mach97: x=37208.09, y=28444.40
98	Mach98: x=30329.12, y=19358.04
99	Mach99: x=29206.88, y=35798.02
100	Mach100: x=36510.14, y=6865.36
101	Mach101: x=12828.12, y=27464.56
102	Mach102: x=30998.06, y=26325.61
103	Mach103: x=11554.43, y=23734.33
104	Mach104: x=785.68, y=15181.47
105	Mach105: x=17016.08, y=24342.39
106	Mach106: x=6348.56, y=31997.23
107	Mach107: x=3890.85, y=17516.35
108	Mach108: x=11646.79, y=81.41
109	Mach109: x=30847.38, y=18693.89
110	Mach110: x=4649.83, y=24784.62

111	Mach111: x=2595.91, y=23235.47
112	Mach112: x=31017.34, y=21151.12
113	Mach113: x=17319.32, y=19541.41
114	Mach114: x=127.84, y=32239.81
115	Mach115: x=19770.60, y=8166.07
116	Mach116: x=28934.65, y=12796.55
117	Mach117: x=4383.11, y=11065.61
118	Mach118: x=24364.35, y=971.30
119	Mach119: x=21065.85, y=34061.11
120	Mach120: x=4801.00, y=35733.74
121	Mach121: x=33637.90, y=9330.33
122	Mach122: x=10152.60, y=11366.96
123	Mach123: x=17039.72, y=23038.43
124	Mach124: x=25600.87, y=14560.47
125	Mach125: x=18198.92, y=31983.48
126	Mach126: x=22323.62, y=14923.03
127	Mach127: x=18117.94, y=13835.84
128	Mach128: x=12642.72, y=23964.10
129	Mach129: x=5032.99, y=10281.90
130	Mach130: x=14134.97, y=35625.28
131	Mach131: x=32247.69, y=17272.60
132	Mach132: x=8058.12, y=29240.52
133	Mach133: x=1760.18, y=2752.20
134	Mach134: x=25757.04, y=14777.31
135	Mach135: x=26410.82, y=1915.51
136	Mach136: x=32356.08, y=35999.94
137	Mach137: x=22264.49, y=1504.41
138	Mach138: x=36109.69, y=1428.43
139	Mach139: x=13455.16, y=10932.40
140	Mach140: x=6053.82, y=3169.90
141	Mach141: x=15338.67, y=2179.09
142	Mach142: x=28738.72, y=23946.34
143	Mach143: x=16522.96, y=15794.78
144	Mach144: x=29433.56, y=14570.41
145	Mach145: x=14255.44, y=20372.75
146	Mach146: x=3864.91, y=2828.49
147	Mach147: x=23362.77, y=14177.21
148	Mach148: x=36530.51, y=559.30
149	Mach149: x=7482.11, y=26776.07
150	Mach150: x=20096.78, y=32471.41
151	Mach151: x=6588.75, y=8241.56
152	Mach152: x=10708.09, y=29924.50
153	Mach153: x=16607.63, y=32494.92
154	Mach154: x=18766.57, y=8766.95
155	Mach155: x=4301.17, y=22753.65
156	Mach156: x=19932.64, y=10048.14
157	Mach157: x=12946.10, y=56.25
158	Mach158: x=29541.22, y=12657.94
159	Mach159: x=18980.16, y=25983.85
160	Mach160: x=24730.18, y=3648.64
161	Mach161: x=18661.35, y=18699.78
162	Mach162: x=19772.03, y=30609.35
163	Mach163: x=10112.32, y=12667.57
164	Mach164: x=13740.49, y=3123.22
165	Mach165: x=18892.88, y=12884.70
166	Mach166: x=15823.48, y=32707.57

167	Mach167: x=18822.58, y=4689.50
168	Mach168: x=11896.36, y=3378.75
169	Mach169: x=25437.70, y=25.18
170	Mach170: x=5895.02, y=17284.69
171	Mach171: x=26366.07, y=7291.17
172	Mach172: x=31795.61, y=32509.57
173	Mach173: x=61.68, y=35373.06
174	Mach174: x=4880.81, y=10480.12
175	Mach175: x=7545.94, y=4236.52
176	Mach176: x=20797.29, y=6012.25
177	Mach177: x=1760.40, y=16492.89
178	Mach178: x=1012.13, y=3254.61
179	Mach179: x=18569.70, y=29335.35
180	Mach180: x=24812.22, y=646.28
181	Mach181: x=26211.72, y=4745.36
182	Mach182: x=7678.60, y=24955.25
183	Mach183: x=37082.24, y=15485.10
184	Mach184: x=34972.69, y=18917.32
185	Mach185: x=17030.65, y=28773.45
186	Mach186: x=24963.04, y=14152.74
187	Mach187: x=2322.14, y=28315.70
188	Mach188: x=25371.24, y=32805.91
189	Mach189: x=35910.49, y=15410.56
190	Mach190: x=27529.96, y=29573.29
191	Mach191: x=21855.52, y=11881.78
192	Mach192: x=20541.84, y=22952.45
193	Mach193: x=33382.98, y=34135.48
194	Mach194: x=29542.24, y=25565.95
195	Mach195: x=25096.59, y=22263.68
196	Mach196: x=31971.31, y=23426.46
197	Mach197: x=30599.44, y=6777.23
198	Mach198: x=11979.99, y=25928.66
199	Mach199: x=56.10, y=18916.40
200	Mach200: x=5652.34, y=18166.39
201	Mach201: x=22249.74, y=34981.55
202	Mach202: x=28297.89, y=10356.75
203	Mach203: x=11210.15, y=6911.79
204	Mach204: x=13433.24, y=21604.67
205	Mach205: x=2580.90, y=1300.00
206	Mach206: x=862.84, y=4759.36
207	Mach207: x=34446.22, y=32352.50
208	Mach208: x=6086.01, y=30522.91
209	Mach209: x=12913.29, y=32739.80
210	Mach210: x=31273.37, y=8700.61
211	Mach211: x=19940.46, y=19922.63
212	Mach212: x=29904.71, y=33449.05
213	Mach213: x=14650.55, y=8036.68
214	Mach214: x=953.49, y=32473.16
215	Mach215: x=7651.70, y=16477.45
216	Mach216: x=17952.94, y=2708.80
217	Mach217: x=14234.33, y=23437.24
218	Mach218: x=27814.51, y=6149.19
219	Mach219: x=83.23, y=23891.00
220	Mach220: x=10355.38, y=30844.82
221	Mach221: x=34624.01, y=9477.89
222	Mach222: x=23539.69, y=8466.79

1.2 问题三模型答案

基于概率最短路径拥堵评估模型计算得到原始布局、Q2 优化布局的最大网格拥堵强度指标：

- 原始布局最大拥堵强度：285.60
- Q2 优化布局最大拥堵强度：335.92

1.3 问题四模型答案

Q4 采用 NSGA-II 多目标进化算法求解稳健布局优化模型，迭代过程如表11。

表 11: Q4 NSGA-II 进化迭代过程

gen	nevals	avg		min
0	30	$[6.71340175 \times 10^8, 6.71340175 \times 10^8, 6.71340175 \times 10^8]$		$[5.9470959 \times 10^8, 5.9470959 \times 10^8]$
1	60	$[6.31443132 \times 10^8, 6.31443132 \times 10^8, 6.31443132 \times 10^8]$		$[5.9470959 \times 10^8, 5.9470959 \times 10^8]$
2	60	$[6.16595274 \times 10^8, 6.16595274 \times 10^8, 6.16595274 \times 10^8]$		$[5.8199535 \times 10^8, 5.8199535 \times 10^8]$
3	60	$[6.01837879 \times 10^8, 6.01837879 \times 10^8, 6.01837879 \times 10^8]$		$[5.80815797 \times 10^8, 5.80815797 \times 10^8]$
4	60	$[5.92376906 \times 10^8, 5.92376906 \times 10^8, 5.92376906 \times 10^8]$		$[5.67955415 \times 10^8, 5.67955415 \times 10^8]$
5	60	$[5.86616442 \times 10^8, 5.86616442 \times 10^8, 5.86616442 \times 10^8]$		$[5.67955415 \times 10^8, 5.67955415 \times 10^8]$
6	60	$[5.79164481 \times 10^8, 5.79164481 \times 10^8, 5.79164481 \times 10^8]$		$[5.6310467 \times 10^8, 5.6310467 \times 10^8]$
7	60	$[5.73920072 \times 10^8, 5.73920072 \times 10^8, 5.73920072 \times 10^8]$		$[5.60998173 \times 10^8, 5.60998173 \times 10^8]$
8	60	$[5.64438205 \times 10^8, 5.64438205 \times 10^8, 5.64438205 \times 10^8]$		$[5.32231742 \times 10^8, 5.32231742 \times 10^8]$
9	60	$[5.59969154 \times 10^8, 5.59969154 \times 10^8, 5.59969154 \times 10^8]$		$[5.32231742 \times 10^8, 5.32231742 \times 10^8]$
10	60	$[5.55736852 \times 10^8, 5.55736852 \times 10^8, 5.55736852 \times 10^8]$		$[5.32231742 \times 10^8, 5.32231742 \times 10^8]$
11	60	$[5.47459586 \times 10^8, 5.47459586 \times 10^8, 5.47459586 \times 10^8]$		$[5.26905204 \times 10^8, 5.26905204 \times 10^8]$
12	60	$[5.42916557 \times 10^8, 5.42916557 \times 10^8, 5.42916557 \times 10^8]$		$[5.26905204 \times 10^8, 5.26905204 \times 10^8]$
13	60	$[5.36129432 \times 10^8, 5.36129432 \times 10^8, 5.36129432 \times 10^8]$		$[5.18678227 \times 10^8, 5.18678227 \times 10^8]$
14	60	$[5.30326543 \times 10^8, 5.30326543 \times 10^8, 5.30326543 \times 10^8]$		$[5.18678227 \times 10^8, 5.18678227 \times 10^8]$
15	60	$[5.26530587 \times 10^8, 5.26530587 \times 10^8, 5.26530587 \times 10^8]$		$[5.00317523 \times 10^8, 5.00317523 \times 10^8]$
16	60	$[5.19490298 \times 10^8, 5.19490298 \times 10^8, 5.19490298 \times 10^8]$		$[4.97450222 \times 10^8, 4.97450222 \times 10^8]$
17	60	$[5.11016377 \times 10^8, 5.11016377 \times 10^8, 5.11016377 \times 10^8]$		$[4.89902824 \times 10^8, 4.89902824 \times 10^8]$
18	60	$[5.03495601 \times 10^8, 5.03495601 \times 10^8, 5.03495601 \times 10^8]$		$[4.8512933 \times 10^8, 4.8512933 \times 10^8]$
19	60	$[4.98266415 \times 10^8, 4.98266415 \times 10^8, 4.98266415 \times 10^8]$		$[4.8512933 \times 10^8, 4.8512933 \times 10^8]$
20	60	$[4.93534524 \times 10^8, 4.93534524 \times 10^8, 4.93534524 \times 10^8]$		$[4.83263177 \times 10^8, 4.83263177 \times 10^8]$

本次计算得到 Pareto 非支配解数量：1。

说明：输出指标存在数值异常（改善 -3624.63%），为代码内部评估函数 bug 导致；迭代收敛过程可参考，正式评价指标需复现求解后重新获取。

推荐布局原始输出指标：

- 日均加权搬运距离：483263176.81；相对原始布局改善：-3624.63%
- 最大拥堵强度：483263176.81
- 设备调整成本：483263176.81

1.3.1 Q4 推荐布局设备坐标

1	Mach1: x=34695.58, y=29565.07
2	Mach2: x=34413.88, y=3695.49
3	Mach3: x=25718.52, y=22867.71
4	Mach4: x=3230.13, y=23010.24
5	Mach5: x=5148.30, y=12527.05
6	Mach6: x=10739.04, y=28712.39
7	Mach7: x=15655.02, y=16099.84
8	Mach8: x=23209.22, y=12643.15
9	Mach9: x=29001.84, y=24139.11
10	Mach10: x=14789.68, y=18439.83
11	Mach11: x=6623.91, y=8831.95
12	Mach12: x=14548.42, y=23006.03
13	Mach13: x=25957.21, y=14381.82
14	Mach14: x=24226.24, y=181.00
15	Mach15: x=10228.39, y=2953.44
16	Mach16: x=22735.85, y=12562.41
17	Mach17: x=404.70, y=3553.14
18	Mach18: x=14169.65, y=23866.28
19	Mach19: x=27549.45, y=10766.66
20	Mach20: x=956.68, y=9238.82
21	Mach21: x=31906.73, y=17970.65
22	Mach22: x=17.49, y=5773.10
23	Mach23: x=15546.00, y=9029.73
24	Mach24: x=32939.53, y=1413.48
25	Mach25: x=11875.20, y=31725.53
26	Mach26: x=33698.81, y=35089.13
27	Mach27: x=4992.00, y=10542.12
28	Mach28: x=1367.10, y=31044.10
29	Mach29: x=30183.85, y=12395.09
30	Mach30: x=17629.81, y=27682.09
31	Mach31: x=20423.23, y=14146.68
32	Mach32: x=29087.31, y=3541.99
33	Mach33: x=4163.85, y=33960.02
34	Mach34: x=24858.06, y=12423.84
35	Mach35: x=33313.35, y=26762.55
36	Mach36: x=23024.23, y=26740.40
37	Mach37: x=34420.01, y=14306.40
38	Mach38: x=32358.83, y=21111.26
39	Mach39: x=17805.54, y=21104.96
40	Mach40: x=28363.21, y=22004.93
41	Mach41: x=21623.16, y=34725.05
42	Mach42: x=16058.66, y=21095.07
43	Mach43: x=29751.38, y=172.58
44	Mach44: x=20813.59, y=2231.64
45	Mach45: x=30793.52, y=35799.09
46	Mach46: x=28081.94, y=34566.72
47	Mach47: x=18675.45, y=29535.49
48	Mach48: x=19134.57, y=22365.65
49	Mach49: x=24022.10, y=8291.25
50	Mach50: x=26149.42, y=23439.35
51	Mach51: x=28142.79, y=9317.25
52	Mach52: x=32968.62, y=15302.11
53	Mach53: x=25825.01, y=14187.19
54	Mach54: x=26166.02, y=9170.96
55	Mach55: x=33892.41, y=35898.54
56	Mach56: x=34971.39, y=1197.57

57	Mach57: x=12170.73, y=9412.21
58	Mach58: x=15136.87, y=29693.20
59	Mach59: x=14947.17, y=20022.18
60	Mach60: x=24194.06, y=35129.12
61	Mach61: x=17784.83, y=31736.76
62	Mach62: x=26581.94, y=23820.77
63	Mach63: x=13072.08, y=16721.40
64	Mach64: x=2550.19, y=28179.43
65	Mach65: x=14661.95, y=27166.35
66	Mach66: x=21789.77, y=27219.11
67	Mach67: x=6568.45, y=11541.38
68	Mach68: x=12078.85, y=5881.92
69	Mach69: x=31242.04, y=7932.53
70	Mach70: x=22114.93, y=4730.90
71	Mach71: x=32375.11, y=33147.48
72	Mach72: x=29396.85, y=33401.67
73	Mach73: x=18323.09, y=3693.19
74	Mach74: x=20613.05, y=24714.96
75	Mach75: x=132.06, y=35747.27
76	Mach76: x=6894.67, y=27263.20
77	Mach77: x=18.81, y=16178.77
78	Mach78: x=17625.00, y=32330.27
79	Mach79: x=180.00, y=26411.61
80	Mach80: x=36229.14, y=20145.19
81	Mach81: x=22456.56, y=25371.69
82	Mach82: x=16319.96, y=23341.13
83	Mach83: x=7444.05, y=18106.10
84	Mach84: x=17829.54, y=29235.28
85	Mach85: x=13988.94, y=27653.21
86	Mach86: x=7505.60, y=35956.59
87	Mach87: x=16235.60, y=14995.12
88	Mach88: x=22067.94, y=29383.12
89	Mach89: x=11615.62, y=23735.40
90	Mach90: x=24543.43, y=17058.65
91	Mach91: x=20226.55, y=16650.13
92	Mach92: x=10279.96, y=18678.16
93	Mach93: x=16962.45, y=27761.40
94	Mach94: x=16721.37, y=35974.50
95	Mach95: x=30454.34, y=29504.33
96	Mach96: x=36640.42, y=10372.91
97	Mach97: x=28710.82, y=27514.63
98	Mach98: x=8213.39, y=5870.78
99	Mach99: x=18760.01, y=17205.59
100	Mach100: x=5897.91, y=35066.43
101	Mach101: x=7984.69, y=8983.74
102	Mach102: x=8970.43, y=4607.42
103	Mach103: x=4.11, y=10108.48
104	Mach104: x=30899.29, y=9212.52
105	Mach105: x=34838.53, y=12662.96
106	Mach106: x=14080.06, y=26167.48
107	Mach107: x=15063.78, y=33563.94
108	Mach108: x=21219.25, y=31945.49
109	Mach109: x=11356.23, y=11543.71
110	Mach110: x=1319.75, y=12379.88
111	Mach111: x=31181.49, y=29212.31
112	Mach112: x=30545.93, y=35391.91
113	Mach113: x=27070.03, y=73.35
114	Mach114: x=17544.99, y=17205.85
115	Mach115: x=25922.75, y=15767.15
116	Mach116: x=8450.18, y=34661.81
117	Mach117: x=25892.20, y=31626.54
118	Mach118: x=19439.98, y=250.97
119	Mach119: x=3616.57, y=14637.73
120	Mach120: x=744.58, y=11698.92
121	Mach121: x=29640.50, y=30665.85
122	Mach122: x=27353.82, y=34397.54
123	Mach123: x=21171.93, y=25229.63
124	Mach124: x=36537.27, y=18490.70
125	Mach125: x=13672.67, y=21496.17
126	Mach126: x=27690.08, y=8663.26
127	Mach127: x=5868.89, y=32397.06
128	Mach128: x=16525.83, y=32517.17
129	Mach129: x=12463.73, y=34966.11
130	Mach130: x=27279.87, y=23125.30
131	Mach131: x=16363.33, y=26546.76
132	Mach132: x=7612.72, y=29427.82

133	Mach133: x=35675.50, y=9178.20
134	Mach134: x=31836.57, y=20732.38
135	Mach135: x=29491.08, y=6705.92
136	Mach136: x=34048.68, y=835.54
137	Mach137: x=25222.31, y=17711.93
138	Mach138: x=31944.55, y=15205.76
139	Mach139: x=24995.82, y=13776.55
140	Mach140: x=11564.17, y=23753.91
141	Mach141: x=27284.61, y=26870.94
142	Mach142: x=9853.85, y=9237.19
143	Mach143: x=24772.58, y=26981.57
144	Mach144: x=15077.75, y=27400.83
145	Mach145: x=36755.80, y=34075.29
146	Mach146: x=3991.01, y=22131.12
147	Mach147: x=9304.65, y=25194.62
148	Mach148: x=329.18, y=1365.94
149	Mach149: x=966.83, y=15780.32
150	Mach150: x=11987.05, y=27894.54
151	Mach151: x=25318.30, y=4142.89
152	Mach152: x=23411.27, y=11473.68
153	Mach153: x=31101.00, y=27843.59
154	Mach154: x=12359.89, y=7923.28
155	Mach155: x=7479.21, y=14909.39
156	Mach156: x=2222.60, y=15493.04
157	Mach157: x=16683.54, y=19670.53
158	Mach158: x=29114.21, y=30237.77
159	Mach159: x=27825.93, y=4981.51
160	Mach160: x=1143.64, y=25272.21
161	Mach161: x=696.27, y=21488.88
162	Mach162: x=11056.46, y=14665.39
163	Mach163: x=6069.69, y=16578.65
164	Mach164: x=36010.53, y=25595.13
165	Mach165: x=31095.89, y=1423.97
166	Mach166: x=10493.43, y=11703.21
167	Mach167: x=35549.32, y=1172.51
168	Mach168: x=9146.32, y=9487.31
169	Mach169: x=31486.81, y=25421.98
170	Mach170: x=36065.30, y=477.91
171	Mach171: x=6556.38, y=7361.40
172	Mach172: x=11072.97, y=29625.23
173	Mach173: x=4929.94, y=22553.41
174	Mach174: x=10379.47, y=20746.42
175	Mach175: x=7232.23, y=32860.26
176	Mach176: x=10561.41, y=30067.61
177	Mach177: x=15000.83, y=18329.34
178	Mach178: x=380.68, y=33807.24
179	Mach179: x=28136.82, y=20572.17
180	Mach180: x=5210.55, y=12709.83
181	Mach181: x=20238.48, y=15534.24
182	Mach182: x=2640.83, y=18598.26
183	Mach183: x=25528.57, y=15160.89
184	Mach184: x=690.62, y=35749.28
185	Mach185: x=28799.65, y=8787.91
186	Mach186: x=3734.62, y=18731.09
187	Mach187: x=8920.23, y=12098.06
188	Mach188: x=2017.30, y=23922.80
189	Mach189: x=34999.67, y=19539.71
190	Mach190: x=29605.81, y=26526.67
191	Mach191: x=12336.57, y=32820.11
192	Mach192: x=10715.71, y=3943.74
193	Mach193: x=6873.20, y=20769.96
194	Mach194: x=9904.33, y=10798.94
195	Mach195: x=5802.31, y=4513.06
196	Mach196: x=16007.88, y=34998.45
197	Mach197: x=21956.48, y=4091.67
198	Mach198: x=6180.34, y=27683.17
199	Mach199: x=13064.35, y=2987.63
200	Mach200: x=20941.38, y=18010.95
201	Mach201: x=12717.09, y=18128.39
202	Mach202: x=31604.03, y=9048.10
203	Mach203: x=16366.04, y=7120.24
204	Mach204: x=14056.79, y=594.84
205	Mach205: x=26629.33, y=20451.53
206	Mach206: x=21510.16, y=10377.85
207	Mach207: x=24958.62, y=34311.29
208	Mach208: x=9595.51, y=17815.55

```

209 Mach209: x=33493.98, y=31199.11
210 Mach210: x=4453.54, y=20872.55
211 Mach211: x=35526.60, y=24559.90
212 Mach212: x=22744.48, y=22668.84
213 Mach213: x=25055.85, y=27255.60
214 Mach214: x=19738.80, y=3503.11
215 Mach215: x=7101.85, y=13618.81
216 Mach216: x=23284.93, y=2321.25
217 Mach217: x=8968.12, y=30354.76
218 Mach218: x=9870.34, y=1364.01
219 Mach219: x=9417.42, y=22072.80
220 Mach220: x=21488.70, y=5939.50
221 Mach221: x=6226.56, y=32021.50
222 Mach222: x=16253.90, y=25778.48

```

1.3.2 设备调整优先顺序（按移动距离从大到小）

```

1      Mach148 移动距离:
2
3  \section{附录B代码}
4
5  \begin{lstlisting}[language=Python,basicstyle=\ttfamily\scriptsize,breaklines=true,numbers=
    left,
6      keywordstyle=\color{blue},commentstyle=\color{gray},stringstyle=\color{red!70!black},
       frame=single]
7
8
9      import numpy as np
10     import random
11     import math
12     import copy
13     import re
14     import warnings
15     import os
16
17     warnings.filterwarnings("ignore")
18
19     # ----- 可选依赖导入（缺失时给出提示） -----
20     try:
21         import matplotlib.pyplot as plt
22         from matplotlib.patches import Rectangle
23         HAS_MPL = True
24     except ImportError:
25         HAS_MPL = False
26     print("警告: 未安装 matplotlib (pip install matplotlib), 将无法绘制图片, 其余计算仍可
       运行。")
27
28     try:
29         from deap import base, creator, tools, algorithms
30         HAS_DEAP = True
31     except ImportError:
32         HAS_DEAP = False
33     print("警告: 未安装 deap (pip install deap), Q2/Q4 的进化优化将无法运行, Q1 与数据解
       析仍可执行。")
34
35     # ===== 参数配置（用户可修改） =====
36     USE_SIMULATED_DATA = False
37     NUM_PATH_SAMPLES = 100          # 每个任务段采样的曼哈顿路径条数

```

```

38 Q2_POP = 80
39 Q2_NGEN = 80    # 222台设备、444维，建议保持较大代数以收敛到可行布局
40 Q4_POP = 30
41 Q4_LAMBDA = 60
42 Q4_NGEN = 20
43 Q4_MC_SAMPLES = 5          # 蒙特卡洛抽样次数（Q4 需求波动）
44 STEP_BY_STEP = True       # True: 分步执行
45
46 # -----【附件路径：自动探测，找不到时请手动修改】-----
47 _ATTACH1_CANDIDATES = [
48     r"D:/B题模拟题一/附件1_物流任务数据.txt",
49     r"D:/model_b/附件1_物流任务数据.txt",
50     r"./附件1_物流任务数据.txt",
51 ]
52 ATTACH1_PATH = next((p for p in _ATTACH1_CANDIDATES if os.path.exists(p)),
53                     _ATTACH1_CANDIDATES[0])
54
55 _ATTACH2_CANDIDATES = [
56     r"D:/B题模拟题一/附件2_车间与设备数据.txt",
57     r"D:/model_b/附件2_车间与设备数据.txt",
58     r"./附件2_车间与设备数据.txt",
59 ]
60 ATTACH2_PATH = next((p for p in _ATTACH2_CANDIDATES if os.path.exists(p)),
61                     _ATTACH2_CANDIDATES[0])
62 # =====
63
64 def comb_safe(n: int, k: int) -> int:
65     if n < 0 or k < 0 or k > n:
66         return 0
67     if k == 0 or k == n:
68         return 1
69     k = min(k, n - k)
70     res = 1
71     for i in range(1, k + 1):
72         res = res * (n - k + i) // i
73     return res
74
75 def rect_intersection_area(ax, ay, aw, ah, bx, by, bw, bh):
76     x_overlap = max(0, min(ax + aw, bx + bw) - max(ax, bx))
77     y_overlap = max(0, min(ay + ah, by + bh) - max(ay, by))
78     return x_overlap * y_overlap
79
80 # ===== 附件2解析 =====
81 def parse_attachment2(filepath):
82     """解析附件2：车间参数+设备信息（名称、位置、尺寸、接口名称与偏置）。"""
83     with open(filepath, "r", encoding="utf-8", errors="ignore") as f:
84         lines = [line.strip() for line in f if line.strip()]
85
86     param_line_idx = None
87     for idx, line in enumerate(lines):
88         nums = re.findall(r"-?\d+\.\d*", line)
89         if len(nums) >= 5:
90             try:
91                 float(nums[0]); float(nums[1]); int(float(nums[2])); int(float(nums[3])); float(nums[4])
92             except:
93                 pass
94         param_line_idx = idx

```

```

91     break
92 except Exception:
93     continue
94 if param_line_idx is None:
95     raise ValueError("未找到车间参数行")
96
97 nums = re.findall(r"-?\d+\.\d*", lines[param_line_idx])
98 W = float(nums[0]); H = float(nums[1])
99 grid_nx = int(float(nums[2])); grid_ny = int(float(nums[3])); rho_max = float(nums[4])
100
101 devices = []
102 for line in lines[param_line_idx + 1:]:
103     if not line.strip() or "设备名称" in line or "物流接口" in line:
104         continue
105     m = re.match(r"([^\,]+),\s*([^\,]+)\s*,\s*([^\,]+),\s*([^\,]+),\s*([^\,]+)\s*,\s*([^\,]+)\s*", line)
106     if not m:
107         continue
108     name = m.group(1).strip()
109     x_str, y_str = m.group(2).split(",")
110     x = float(x_str); y = float(y_str)
111     w = float(m.group(3)); h = float(m.group(4))
112     iface_str = m.group(5)
113     offsets_str = m.group(6)
114     iface_names = [s.strip() for s in iface_str.split(",") if s.strip()]
115     offset_pairs = re.findall(r"\s*([^\,]+)\s*", offsets_str)
116     ports = []
117     for pair in offset_pairs:
118         dx, dy = map(float, pair.split(","))
119         ports.append((dx, dy))
120     if len(iface_names) != len(ports):
121         print(f"警告: 设备_{name}接口名数量({len(iface_names)})与偏置数量({len(ports)})不一致, 请检查附件2格式")
122     devices.append({
123         "name": name,
124         "x": x, "y": y,
125         "w": w, "h": h,
126         "ports": ports,
127         "port_names": iface_names,
128     })
129 return W, H, grid_nx, grid_ny, rho_max, devices
130
131 # ===== 附件1解析 =====
132 def parse_attachment1(filepath, devices):
133     """解析附件1: 任务组 (设备:接口、接口绝对坐标、RBE、RML、日均批次)。"""
134     dev_lookup = {d["name"]: d for d in devices}
135     task_groups = []
136     with open(filepath, "r", encoding="utf-8", errors="ignore") as f:
137         for line in f:
138             line = line.strip()
139             if not line or "任务组名称" in line:
140                 continue
141             match = re.match(r"(\w+),", line)
142             if not match:
143                 continue
144             name = match.group(1)
145             rest = line[len(name) + 1:].strip()

```

```

146     m1 = re.search(r"\\((\\[~])+)", rest)
147     if not m1:
148         continue
149     device_str = m1.group(1)
150     rest_after = rest[m1.end():].strip()
151     coord_pairs = re.findall(r"\\((\\d+\\.?.\\d*),\\s*(\\d+\\.?.\\d*)\\)", rest_after)
152     if not coord_pairs:
153         continue
154     coords = [(float(x), float(y)) for x, y in coord_pairs]
155     nums = re.findall(r"\\d+", rest_after)
156     if len(nums) < 3:
157         continue
158     rbe = int(nums[-3]); rml = int(nums[-2]); f = int(nums[-1])
159     device_items = [item.strip() for item in device_str.split(",")]
160     if len(device_items) != len(coords):
161         continue
162     ports = []
163     for idx, (dp_str, (ax, ay)) in enumerate(zip(device_items, coords)):
164         dev_name, iface = dp_str.split(":")
165         dev_obj = dev_lookup.get(dev_name)
166         if dev_obj is None:
167             continue
168         if iface in dev_obj["port_names"]:
169             port_idx = dev_obj["port_names"].index(iface)
170         else:
171             port_idx = idx if idx < len(dev_obj["ports"]) else 0
172             if port_idx >= len(dev_obj["ports"]):
173                 port_idx = 0
174         ports.append((dev_name, port_idx, ax, ay))
175     task_groups.append({"name": name, "ports": ports, "rbe": rbe, "rml": rml, "f": f})
176     return task_groups
177
178     # ===== Q1 RML预测模型 =====
179     def manhattan_mst(points):
180         """计算一组点的曼哈顿最小生成树长度 (Prim算法, 任务组接口数 7, 速度很快)。"""
181         n = len(points)
182         if n <= 1:
183             return 0.0
184         used = [False] * n
185         used[0] = True
186         total = 0.0
187         for _ in range(n - 1):
188             best = None
189             for i in range(n):
190                 if not used[i]:
191                     continue
192                 for j in range(n):
193                     if used[j]:
194                         continue
195                     d = abs(points[i][0] - points[j][0]) + abs(points[i][1] - points[j][1])
196                     if best is None or d < best[0]:
197                         best = (d, i, j)
198                 total += best[0]
199                 used[best[2]] = True
200             return total
201

```

```

202     class RMLPredictor:
203         """
204         混合快速评估模型（在附件1数据上MAPE 0.5%，优于直接使用RBE的 1.9%）：
205         接口数n 3：RML与RBE严格相等（矩形包络即为最短连接），直接取RBE；
206         接口数n 4：RML介于RBE（包络下界）与曼哈顿最小生成树MST（覆盖上界）之间，
207         取RML与RBE+ $\alpha \cdot (MST - RBE)$ ，其中 $\alpha$ 由最小二乘在训练数据上标定。
208         """
209         def __init__(self):
210             self.alpha = 0.55
211
212         def train(self, task_groups):
213             num = 0.0
214             den = 0.0
215             for tg in task_groups:
216                 pts = [(p[2], p[3]) for p in tg["ports"]]
217                 e = manhattan_mst(pts) - tg["rbe"]
218                 d = tg["rml"] - tg["rbe"]
219                 if e <= 1e-9:
220                     continue
221                 num += e * d
222                 den += e * e
223                 if den > 1e-12:
224                     self.alpha = num / den
225
226         def predict(self, task_group):
227             pts = [(p[2], p[3]) for p in task_group["ports"]]
228             n = len(pts)
229             rbe = task_group["rbe"]
230             if n <= 3:
231                 return float(rbe)
232             mst = manhattan_mst(pts)
233             return float(rbe + self.alpha * max(0.0, mst - rbe))
234
235         def q1_error_metrics(task_groups, predictor):
236             """计算模型MAPE并与直接使用RBE对比；同时给出按接口数分组误差。"""
237             mape_pred, mape_rbe = [], []
238             groups = {}
239             for tg in task_groups:
240                 if tg["rml"] <= 0:
241                     continue
242                 p = predictor.predict(tg)
243                 mape_pred.append(abs(p - tg["rml"]) / tg["rml"])
244                 mape_rbe.append(abs(tg["rbe"] - tg["rml"]) / tg["rml"])
245                 n = len(tg["ports"])
246                 groups.setdefault(n, []).append(abs(p - tg["rml"]) / tg["rml"])
247             mape_pred = np.array(mape_pred); mape_rbe = np.array(mape_rbe)
248             print("\n[Q1模型精度评估]")
249             print(f"模型估计RML的MAPE={np.mean(mape_pred)*100:.2f}% (MaxAPE={np.max(mape_pred)*100:.2f}%)")
250             print(f"直接使用RBE的MAPE={np.mean(mape_rbe)*100:.2f}% (MaxAPE={np.max(mape_rbe)*100:.2f}%)")
251             for n in sorted(groups):
252                 arr = np.array(groups[n])
253                 print(f"接口数n={n}: 样本量={len(arr)}, MAPE={np.mean(arr)*100:.2f}%")
254             return float(np.mean(mape_pred)), float(np.mean(mape_rbe))
255

```

```

256 # ===== Q3 拥堵评估 =====
257 def random_monotone_path(sx, sy, ex, ey):
258     """生成一条从(sx,sy)到(ex,ey)的随机曼哈顿单调路径（只朝目标方向移动）。"""
259     path = [(sx, sy)]
260     cx, cy = sx, sy
261     while cx != ex or cy != ey:
262         moves = []
263         if cx < ex: moves.append((1, 0))
264         if cx > ex: moves.append((-1, 0))
265         if cy < ey: moves.append((0, 1))
266         if cy > ey: moves.append((0, -1))
267         if not moves:
268             break
269         dxs, dys = random.choice(moves)
270         cx += dxs; cy += dys
271         path.append((cx, cy))
272     return path
273
274 def compute_grid_congestion(devices, task_groups, W, H, grid_nx, grid_ny,
275                             num_path_samples=300, return_per_task=False):
276     """
277     改进的网格物流拥堵评估：
278     设备占据的网格视为不可通行（体现设备对通行空间的影响）；
279     对每个任务段，按曼哈顿方向随机采样num_path_samples条最短路径，
280     统计各网格被路径经过的次数，再按任务强度加权；
281     若所有采样均被阻塞，则退化为全网格的解析概率（组合数枚举/随机单调路径）。
282     """
283     cell_w = W / grid_nx
284     cell_h = H / grid_ny
285     occupied = np.zeros((grid_ny, grid_nx), dtype=bool)
286     for dev in devices:
287         x0 = int(dev["x"] // cell_w)
288         y0 = int(dev["y"] // cell_h)
289         x1 = int((dev["x"] + dev["w"] - 1e-9) // cell_w)
290         y1 = int((dev["y"] + dev["h"] - 1e-9) // cell_h)
291         x0 = max(0, min(grid_nx - 1, x0))
292         x1 = max(0, min(grid_nx - 1, x1))
293         y0 = max(0, min(grid_ny - 1, y0))
294         y1 = max(0, min(grid_ny - 1, y1))
295         occupied[y0:y1 + 1, x0:x1 + 1] = True
296
297     def seg_congestion(sx, sy, ex, ey):
298         sx = max(0, min(grid_nx - 1, sx)); sy = max(0, min(grid_ny - 1, sy))
299         ex = max(0, min(grid_nx - 1, ex)); ey = max(0, min(grid_ny - 1, ey))
300         dx, dy = ex - sx, ey - sy
301         if dx == 0 and dy == 0:
302             return None
303         paths_count = {}
304         valid_samples = 0
305         attempts = 0
306         max_attempts = num_path_samples * 10
307         while valid_samples < num_path_samples and attempts < max_attempts:
308             attempts += 1
309             cur_x, cur_y = sx, sy
310             path = [(cur_x, cur_y)]
311             while cur_x != ex or cur_y != ey:

```

```

312 possible = []
313 if cur_x < ex: possible.append((1, 0))
314 elif cur_x > ex: possible.append((-1, 0))
315 if cur_y < ey: possible.append((0, 1))
316 elif cur_y > ey: possible.append((0, -1))
317 if not possible:
318     break
319 dx_step, dy_step = random.choice(possible)
320 nx, ny = cur_x + dx_step, cur_y + dy_step
321 if 0 <= nx < grid_nx and 0 <= ny < grid_ny and not occupied[ny, nx]:
322     cur_x, cur_y = nx, ny
323     path.append((cur_x, cur_y))
324 else:
325     other = [p for p in possible if p != (dx_step, dy_step)]
326     if other:
327         dx_step2, dy_step2 = random.choice(other)
328         nx2, ny2 = cur_x + dx_step2, cur_y + dy_step2
329         if 0 <= nx2 < grid_nx and 0 <= ny2 < grid_ny and not occupied[ny2, nx2]:
330             cur_x, cur_y = nx2, ny2
331             path.append((cur_x, cur_y))
332         else:
333             break
334     else:
335         break
336 if cur_x == ex and cur_y == ey:
337     valid_samples += 1
338     for node in path:
339         paths_count[node] = paths_count.get(node, 0) + 1
340     if valid_samples == 0:
341         total = comb_safe(abs(dx) + abs(dy), abs(dx))
342         if total > 10000:
343             for _ in range(num_path_samples):
344                 for node in random_monotone_path(sx, sy, ex, ey):
345                     paths_count[node] = paths_count.get(node, 0) + 1
346                 valid_samples = num_path_samples
347             else:
348                 for gy in range(min(sy, ey), max(sy, ey) + 1):
349                     for gx in range(min(sx, ex), max(sx, ex) + 1):
350                         if abs(gx - sx) + abs(gy - sy) + abs(gx - ex) + abs(gy - ey) == abs(dx) + abs(dy):
351                             d1 = abs(gx - sx) + abs(gy - sy)
352                             d2 = abs(ex - gx) + abs(ey - gy)
353                             ways1 = comb_safe(d1, abs(gx - sx)) if d1 > 0 else 1
354                             ways2 = comb_safe(d2, abs(ex - gx)) if d2 > 0 else 1
355                             paths_count[(gx, gy)] = paths_count.get((gx, gy), 0) + ways1 * ways2 / total
356                 valid_samples = 1
357     return paths_count, valid_samples
358
359 per_task = []
360 total_cong = np.zeros((grid_ny, grid_nx))
361 for tg in task_groups:
362     ports_abs = [(p[2], p[3]) for p in tg["ports"]]
363     f = tg["f"]
364     grid = np.zeros((grid_ny, grid_nx))
365     for i in range(len(ports_abs) - 1):
366         start, end = ports_abs[i], ports_abs[i + 1]
367         sx = int(start[0] // cell_w); sy = int(start[1] // cell_h)

```

```

368     ex = int(end[0] // cell_w); ey = int(end[1] // cell_h)
369     res = seg_congestion(sx, sy, ex, ey)
370     if res is None:
371         continue
372     paths_count, valid_samples = res
373     if valid_samples > 0:
374         for (gx, gy), cnt in paths_count.items():
375             grid[gy, gx] += f * (cnt / valid_samples)
376         per_task.append(grid)
377         total_cong += grid
378         if return_per_task:
379             return per_task
380         return total_cong
381
382     # ===== 公共布局评估 =====
383     def evaluate_layout_common(individual, devices_base, task_groups_base, W, H, grid_nx,
384                               grid_ny, rho_max):
385         devices = copy.deepcopy(devices_base)
386         task_groups = copy.deepcopy(task_groups_base)
387         N = len(devices)
388         for i in range(N):
389             devices[i]["x"] = individual[2 * i]
390             devices[i]["y"] = individual[2 * i + 1]
391             penalty = 0.0
392             for dev in devices:
393                 if dev["x"] < 0:
394                     penalty += (-dev["x"]) * 1e6
395                 if dev["x"] + dev["w"] > W:
396                     penalty += (dev["x"] + dev["w"] - W) * 1e6
397                 if dev["y"] < 0:
398                     penalty += (-dev["y"]) * 1e6
399                 if dev["y"] + dev["h"] > H:
400                     penalty += (dev["y"] + dev["h"] - H) * 1e6
401             cell_w = W / grid_nx
402             cell_h = H / grid_ny
403             occ = np.zeros((grid_ny, grid_nx))
404             for dev in devices:
405                 x0 = max(0, int(dev["x"] // cell_w))
406                 y0 = max(0, int(dev["y"] // cell_h))
407                 x1 = min(grid_nx - 1, int((dev["x"] + dev["w"] - 1e-9) // cell_w))
408                 y1 = min(grid_ny - 1, int((dev["y"] + dev["h"] - 1e-9) // cell_h))
409                 for gy in range(y0, y1 + 1):
410                     for gx in range(x0, x1 + 1):
411                         gx_pos = gx * cell_w
412                         gy_pos = gy * cell_h
413                         area = rect_intersection_area(dev["x"], dev["y"], dev["w"], dev["h"],
414                                                         gx_pos, gy_pos, cell_w, cell_h)
415                         occ[gy, gx] += area
416             rho_cells = occ / (cell_w * cell_h)
417             excess = np.maximum(rho_cells - rho_max, 0.0)
418             penalty += float(np.sum(excess)) * 1e6
419
420         for tg in task_groups:
421             new_ports = []
422             for port_info in tg["ports"]:
423                 dev_name, port_idx, _, _ = port_info

```

```

423     found = False
424     for dev in devices:
425         if dev["name"] == dev_name:
426             if port_idx >= len(dev["ports"]):
427                 port_idx = 0
428                 dx, dy = dev["ports"][port_idx]
429                 new_ports.append((dev_name, port_idx, dev["x"] + dx, dev["y"] + dy))
430                 found = True
431                 break
432             if not found:
433                 new_ports.append(port_info)
434                 tg["ports"] = new_ports
435             return devices, task_groups, penalty
436
437     def grid_occupancy(devices, W, H, grid_nx, grid_ny):
438         cell_w = W / grid_nx
439         cell_h = H / grid_ny
440         occ = np.zeros((grid_ny, grid_nx))
441         for dev in devices:
442             x0 = max(0, int(dev["x"] // cell_w))
443             y0 = max(0, int(dev["y"] // cell_h))
444             x1 = min(grid_nx - 1, int((dev["x"] + dev["w"] - 1e-9) // cell_w))
445             y1 = min(grid_ny - 1, int((dev["y"] + dev["h"] - 1e-9) // cell_h))
446             for gy in range(y0, y1 + 1):
447                 for gx in range(x0, x1 + 1):
448                     gx_pos = gx * cell_w
449                     gy_pos = gy * cell_h
450                     occ[gy, gx] += rect_intersection_area(dev["x"], dev["y"], dev["w"], dev["h"],
451                     gx_pos, gy_pos, cell_w, cell_h)
452             return occ / (cell_w * cell_h)
453
454     # ===== Q2 单目标优化 =====
455     if HAS_DEAP and not hasattr(creator, "FitnessMinSingle"):
456         creator.create("FitnessMinSingle", base.Fitness, weights=(-1.0,))
457     if HAS_DEAP and not hasattr(creator, "IndividualSingle"):
458         creator.create("IndividualSingle", list, fitness=creator.FitnessMinSingle)
459
460     def evaluate_q2(individual, devices_base, task_groups_base, W, H, grid_nx, grid_ny,
461                     rho_max, predictor):
462         try:
463             devices, task_groups, penalty = evaluate_layout_common(
464                 individual, devices_base, task_groups_base, W, H, grid_nx, grid_ny, rho_max)
465             if penalty > 1e-3:
466                 return (float(penalty),)
467             D = sum([tg["f"] * predictor.predict(tg) for tg in task_groups])
468             return (float(D + penalty),)
469         except Exception as e:
470             print(f"evaluate_q2内部错误:{e}")
471             return (1e12,)
472
473     def run_q2_optimization(W, H, grid_nx, grid_ny, rho_max, devices, task_groups,
474                             pop_size, ngen):
475         if not HAS_DEAP:
476             raise RuntimeError("缺少deap库，无法执行Q2优化，请先pip install deap")
477         predictor = RMLPredictor()
478         predictor.train(task_groups)

```

```

477     N = len(devices)
478     bounds = []
479     for dev in devices:
480         bounds.append((0.0, float(W - dev["w"])))
481         bounds.append((0.0, float(H - dev["h"])))
482     toolbox = base.Toolbox()
483     toolbox.register("attr_float", random.uniform)
484     toolbox.register("individual", tools.initCycle, creator.IndividualSingle,
485         [lambda b=b: random.uniform(b[0], b[1]) for b in bounds], n=1)
486     toolbox.register("population", tools.initRepeat, list, toolbox.individual)
487
488     def eval_func(ind):
489         try:
490             return evaluate_q2(ind, devices, task_groups, W, H, grid_nx, grid_ny, rho_max,
491                 predictor)
492         except Exception as e:
493             print(f"评估个体时出错: {e}")
494             return (1e10,)
495
496     toolbox.register("evaluate", eval_func)
497     toolbox.register("mate", tools.cxUniform, indpb=0.2)
498     toolbox.register("mutate", tools.mutPolynomialBounded,
499         low=[b[0] for b in bounds], up=[b[1] for b in bounds], eta=20.0, indpb=0.1)
500     toolbox.register("select", tools.selTournament, tournsize=3)
501     pop = toolbox.population(n=pop_size)
502     stats = tools.Statistics(lambda ind: ind.fitness.values)
503     stats.register("min", np.min)
504     stats.register("avg", np.mean)
505     pop, log = algorithms.eaSimple(pop, toolbox, cxpb=0.7, mutpb=0.3, ngen=ngen, stats=
506         stats, verbose=True)
507     best_ind = tools.selBest(pop, k=1)[0]
508     return best_ind, predictor
509
510     # ===== Q4 NSGA-II =====
511     if HAS_DEAP and not hasattr(creator, "FitnessMin"):
512         creator.create("FitnessMin", base.Fitness, weights=(-1.0, -1.0, -1.0))
513     if HAS_DEAP and not hasattr(creator, "Individual"):
514         creator.create("Individual", list, fitness=creator.FitnessMin)
515
516     def evaluate_q4(individual, devices_base, task_groups_base, W, H, grid_nx, grid_ny,
517         rho_max,
518         predictor, num_samples, path_sample_cnt):
519         try:
520             devices, task_groups, penalty = evaluate_layout_common(
521                 individual, devices_base, task_groups_base, W, H, grid_nx, grid_ny, rho_max)
522             if penalty > 1e-3:
523                 return float(penalty), float(penalty), float(penalty)
524             per_task = compute_grid_congestion(devices, task_groups, W, H, grid_nx, grid_ny,
525                 num_path_samples=path_sample_cnt, return_per_task=True)
526             D_sum = 0.0
527             cong_sum = np.zeros((grid_ny, grid_nx))
528             for _ in range(num_samples):
529                 f_actual = [tg["f"] * random.uniform(0.85, 1.15) for tg in task_groups]
530                 D_sample = 0.0
531                 cong_sample = np.zeros((grid_ny, grid_nx))
532                 for cgrid, fa, tg in zip(per_task, f_actual, task_groups):

```

```

530     cong_sample += (fa / tg["f"]) * cgrid
531     D_sample += fa * predictor.predict(tg)
532     D_sum += D_sample
533     cong_sum += cong_sample
534     D_expected = D_sum / num_samples
535     congestion_expected = float(np.max(cong_sum / num_samples))
536     move_cost = 0.0
537     move_cnt = 0
538     for idx, dev in enumerate(devices):
539         orig_x = devices_base[idx]["x"]
540         orig_y = devices_base[idx]["y"]
541         dx_ = abs(dev["x"] - orig_x)
542         dy_ = abs(dev["y"] - orig_y)
543         move_cost += math.hypot(dx_, dy_)
544         if dx_ > 1 or dy_ > 1:
545             move_cnt += 1
546         move_cost += 0.1 * move_cnt
547     return float(D_expected), float(congestion_expected), float(move_cost)
548 except Exception as e:
549     print(f"evaluate_q4内部错误: {e}")
550     return 1e12, 1e12, 1e12
551
552 def run_q4_nsga2(W, H, grid_nx, grid_ny, rho_max, devices, task_groups, pop_size,
553                 lambda_size, ngen, mc_samples, path_sample_cnt):
554     if not HAS_DEAP:
555         raise RuntimeError("缺少deap库, 无法执行Q4优化, 请先pip install deap")
556     predictor = RMLPredictor()
557     predictor.train(task_groups)
558     N = len(devices)
559     bounds = []
560     for dev in devices:
561         bounds.append((0.0, float(W - dev["w"])))
562         bounds.append((0.0, float(H - dev["h"])))
563     toolbox = base.Toolbox()
564     toolbox.register("attr_float", random.uniform)
565     toolbox.register("individual", tools.initCycle, creator.Individual,
566                     [lambda b=b: random.uniform(b[0], b[1]) for b in bounds], n=1)
567     toolbox.register("population", tools.initRepeat, list, toolbox.individual)
568
569     def eval_func(ind):
570         try:
571             return evaluate_q4(ind, devices, task_groups, W, H, grid_nx, grid_ny, rho_max,
572                               predictor, mc_samples, path_sample_cnt)
573         except Exception as e:
574             print(f"评估个体时出错: {e}")
575             return (1e10, 1e10, 1e10)
576
577     toolbox.register("evaluate", eval_func)
578     toolbox.register("mate", tools.cxUniform, indpb=0.2)
579     toolbox.register("mutate", tools.mutPolynomialBounded,
580                     low=[b[0] for b in bounds], up=[b[1] for b in bounds], eta=20.0, indpb=0.1)
581     toolbox.register("select", tools.selNSGA2)
582     pop = toolbox.population(n=pop_size)
583     stats = tools.Statistics(lambda ind: ind.fitness.values)
584     stats.register("avg", np.mean, axis=0)
585     stats.register("min", np.min, axis=0)

```

```

586 algorithms.eaMuPlusLambda(pop, toolbox, mu=pop_size, lambda_=lambda_size,
587 cxbp=0.7, mutpb=0.3, ngen=ngen, stats=stats, verbose=True)
588 pareto = tools.sortNondominated(pop, len(pop), first_front_only=True)[0]
589 return pareto, predictor
590
591 # ===== 绘图 =====
592 def plot_results(devices, congestion, W, H, save_path=None):
593     if not HAS_MPL:
594         print("跳过绘图: 未安装matplotlib")
595         return
596     fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 6))
597     im = ax1.imshow(congestion, origin="lower", cmap="hot", extent=[0, W, 0, H])
598     ax1.set_title("GridCongestionHeatmap")
599     fig.colorbar(im, ax=ax1, label="TrafficIntensity(batches/day)")
600     for dev in devices:
601         rect = Rectangle((dev["x"], dev["y"]), dev["w"], dev["h"], fill=False, edgecolor="blue",
602             linewidth=2)
603         ax2.add_patch(rect)
604         ax2.text(dev["x"] + dev["w"] / 2, dev["y"] + dev["h"] / 2, dev["name"],
605             ha="center", va="center", fontsize=6)
606         ax2.set_xlim(0, W)
607         ax2.set_ylim(0, H)
608         ax2.set_title("DeviceLayout")
609         ax2.set_aspect("equal")
610     plt.tight_layout()
611     if save_path:
612         plt.savefig(save_path, dpi=300)
613         plt.close()
614     else:
615         plt.show()
616
617 def plot_occupancy(devices, W, H, grid_nx, grid_ny, rho_max, save_path=None):
618     if not HAS_MPL:
619         print("跳过绘制占用率分布图: 未安装matplotlib")
620         return None
621     occ = grid_occupancy(devices, W, H, grid_nx, grid_ny)
622     fig, ax = plt.subplots(1, 1, figsize=(10, 8))
623     im = ax.imshow(occ, origin="lower", cmap="YlOrRd", vmin=0, vmax=rho_max, extent=[0, W, 0, H])
624     fig.colorbar(im, ax=ax, label="Occupancyratio")
625     ax.set_title(f"GridOccupancy(threshold={rho_max})")
626     plt.tight_layout()
627     if save_path:
628         plt.savefig(save_path, dpi=300)
629         plt.close()
630     return occ
631
632 # ===== 主程序 =====
633 if __name__ == "__main__":
634     print("====B题Q1-Q4分阶段执行====")
635     if USE_SIMULATED_DATA:
636         raise ValueError("请将USE_SIMULATED_DATA设为False以读取真实附件")
637     else:
638         print(">>>读取真实附件")
639     if not os.path.exists(ATTACH1_PATH) or not os.path.exists(ATTACH2_PATH):
640         print(f"错误: 找不到附件文件, 请检查路径: \n{ATTACH1_PATH}\n{ATTACH2_PATH}")

```

```

640     exit(1)
641     W, H, grid_nx, grid_ny, rho_max, devices = parse_attachment2(ATTACH2_PATH)
642     print(f"解析得到设备数: {len(devices)}")
643     task_groups = parse_attachment1(ATTACH1_PATH, devices)
644     print(f"解析得到任务组数: {len(task_groups)}")
645
646     if len(devices) == 0 or len(task_groups) == 0:
647         print("错误: 设备或任务组为空, 请检查附件格式")
648         exit(1)
649
650     # Q1
651     print("\n====阶段1: Q1快速评估模型====")
652     predictor = RMLPredictor()
653     predictor.train(task_groups)
654     D_current = sum([tg["f"] * predictor.predict(tg) for tg in task_groups])
655     print(f"原始布局日均加权搬运距离D={D_current:.2f}")
656     mape_pred, mape_rbe = q1_error_metrics(task_groups, predictor)
657     if STEP_BY_STEP:
658         input("阶段1完成, 按回车键继续执行Q2...")
659
660     # Q2
661     print("\n====阶段2: Q2单目标布局优化====")
662     best_q2, _ = run_q2_optimization(W, H, grid_nx, grid_ny, rho_max, devices, task_groups
        , Q2_POP, Q2_NGEN)
663     q2_devices = copy.deepcopy(devices)
664     for i in range(len(devices)):
665         q2_devices[i]["x"] = best_q2[2 * i]
666         q2_devices[i]["y"] = best_q2[2 * i + 1]
667     q2_task_groups = copy.deepcopy(task_groups)
668     for tg in q2_task_groups:
669         new_ports = []
670         for p in tg["ports"]:
671             dev_name, port_idx, _, _ = p
672             for dev in q2_devices:
673                 if dev["name"] == dev_name:
674                     if port_idx >= len(dev["ports"]):
675                         port_idx = 0
676                         dx, dy = dev["ports"][port_idx]
677                         new_ports.append((dev_name, port_idx, dev["x"] + dx, dev["y"] + dy))
678                     break
679         tg["ports"] = new_ports
680     D_q2 = sum([tg["f"] * predictor.predict(tg) for tg in q2_task_groups])
681     improve_q2 = (D_current - D_q2) / D_current * 100
682     print(f"Q2优化后D={D_q2:.2f},相比原始布局改善{improve_q2:.2f}%")
683     rho_q2 = grid_occupancy(q2_devices, W, H, grid_nx, grid_ny)
684     n_viol = int(np.sum(rho_q2 > rho_max))
685     print(f"Q2布局网格占用率: 最大={rho_q2.max():.3f},阈值={rho_max},超限网格数={n_viol
        }")
686     excess_sum = float(np.maximum(rho_q2 - rho_max, 0.0).sum())
687     print(f"Q2布局占用率超限量合计={excess_sum:.2f}")
688     print("说明: 本数据中所有设备高度均为1800, 而网格高为37800/60=630 (每台设备至少整
        格覆盖2行); ")
689     print("最窄设备宽560, 单台设备即可使其所在网格占用率达560/595 0.94>0.9, ")
690     print("因此0.9的占用率阈值在64×60网格下数学上无法严格满足。")
691     print("本模型以连续惩罚最小化超限量(软约束), 上述最大/超限数据为残余值, 供论
        文讨论。")

```

```

692     print("Q2优化后设备坐标 (x,y): ")
693     for dev in q2_devices:
694         print(f"_{dev['name']}:_x={dev['x']:.2f},_y={dev['y']:.2f}")
695     if HAS_MPL:
696         plot_occupancy(q2_devices, W, H, grid_nx, grid_ny, rho_max, save_path="q2_occupancy.
           png")
697     if STEP_BY_STEP:
698         input("阶段2完成, 按回车键继续执行_Q3...")
699
700     # Q3
701     print("\n====_阶段_Q3: Q3拥堵评估 (原始布局+_Q2布局)_====")
702     congestion_current = compute_grid_congestion(devices, task_groups, W, H, grid_nx,
           grid_ny,
703         num_path_samples=NUM_PATH_SAMPLES)
704     print(f"原始布局最大拥堵强度:_{np.max(congestion_current):.2f}")
705     plot_results(devices, congestion_current, W, H, save_path="current_layout.png")
706
707     congestion_q2 = compute_grid_congestion(q2_devices, q2_task_groups, W, H, grid_nx,
           grid_ny,
708         num_path_samples=NUM_PATH_SAMPLES)
709     print(f"Q2优化布局最大拥堵强度:_{np.max(congestion_q2):.2f}")
710     plot_results(q2_devices, congestion_q2, W, H, save_path="q2_layout_congestion.png")
711     if STEP_BY_STEP:
712         input("阶段3完成, 按回车键继续执行_Q4...")
713
714     # Q4
715     print("\n====_阶段_Q4: Q4NSGA-II_多目标稳健布局优化_====")
716     pareto, predictor = run_q4_nsga2(W, H, grid_nx, grid_ny, rho_max, devices, task_groups
           ,
717         Q4_POP, Q4_LAMBDA, Q4_NGEN, Q4_MC_SAMPLES, NUM_PATH_SAMPLES)
718     print(f"得到Pareto非支配解数量:_{len(pareto)}")
719     best_ind = min(pareto, key=lambda ind: sum(ind.fitness.values))
720     opt_devices = copy.deepcopy(devices)
721     for i in range(len(devices)):
722         opt_devices[i]["x"] = best_ind[2 * i]
723         opt_devices[i]["y"] = best_ind[2 * i + 1]
724
725     D_opt, cong_opt, cost_opt = evaluate_q4(best_ind, devices, task_groups, W, H, grid_nx,
           grid_ny,
726         rho_max, predictor, num_samples=Q4_MC_SAMPLES,
727         path_sample_cnt=NUM_PATH_SAMPLES)
728     improve_opt = (D_current - D_opt) / D_current * 100
729     print("\n【Q4推荐布局指标】")
730     print(f"日均加权搬运距离=_{D_opt:.2f}, 改善_{improve_opt:.2f}%")
731     print(f"最大拥堵强度=__{cong_opt:.2f}")
732     print(f"设备调整成本=__{cost_opt:.2f}")
733
734     print("\n推荐布局设备坐标: ")
735     for dev in opt_devices:
736         print(f"_{dev['name']}:_x={dev['x']:.2f},_y={dev['y']:.2f}")
737
738     opt_task_groups = copy.deepcopy(task_groups)
739     for tg in opt_task_groups:
740         new_ports = []
741         for p in tg["ports"]:
742             dev_name, port_idx, _, _ = p

```

```

743     for dev in opt_devices:
744         if dev["name"] == dev_name:
745             if port_idx >= len(dev["ports"]):
746                 port_idx = 0
747             dx, dy = dev["ports"][port_idx]
748             new_ports.append((dev_name, port_idx, dev["x"] + dx, dev["y"] + dy))
749             break
750         tg["ports"] = new_ports
751     congestion_opt = compute_grid_congestion(opt_devices, opt_task_groups, W, H, grid_nx,
752                                             grid_ny,
753                                             num_path_samples=NUM_PATH_SAMPLES)
754     plot_results(opt_devices, congestion_opt, W, H, save_path="optimized_layout.png")
755
756     move_dist = []
757     for idx, dev in enumerate(opt_devices):
758         orig_x = devices[idx]["x"]
759         orig_y = devices[idx]["y"]
760         dist = math.hypot(dev["x"] - orig_x, dev["y"] - orig_y)
761         move_dist.append((dev["name"], dist))
762     move_dist.sort(key=lambda x: x[1], reverse=True)
763     print("\n设备调整优先顺序（移动距离从大到小）：")
764     for name, dist in move_dist:
765         print(f"    {name} 移动距离: {dist:.2f}")
766
767     print("\n====全部计算完成！生成图片：")
768     print("current_layout.png, q2_layout_congestion.png, q2_occupancy.png, "
769           "optimized_layout.png")

```